

Multi-Disaster Emergency Response System Using Machine Learning

A project report submitted in partial fulfillment of the requirements

for the award of credits in

Machine Learning Lab

A skill-oriented course of

Bachelor of Technology

in

ELECTRONICS & COMMUNICATION ENGINEERING

By

Sk. Abdul Rajak

23BQ1A04E4

T. Kushwanth

24BQ5A0420

V. Jaswanth

23BQ1A04H3



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NBA and NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

APRIL 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY: KAKINADA



CERTIFICATE

This is to certify that the project titled “**Multi-Disaster Emergency Response System Using Machine Learning**” is a bonafide record of work done by **Sk. Abdul Rajak, T. Kushwanth and V. Jaswanth** under the guidance of **Mrs.M Ramya Harika, Assistant Professor** in partial fulfillment of the requirement for the award of credits in **Machine Learning Lab-** a skill-oriented course of Bachelor of Technology in Electronics and Communication Engineering, JNTUK during the academic year 2025–26.

Mrs.M Ramya Harika
Course Instructor

Dr. Sk.Enaul Haq
Head of the Department

DECLARATION

We are **Sk. Abdul Rajak, T. Kushwanth and V. Jashwanth** hereby declare that the project report entitled “**Multi-Disaster Emergency Response System Using Machine Learning**” done by us under the guidance of **Mrs. M Ramya Harika, Assistant Professor, Department of Electronics and Communication Engineering** is submitted by partial fulfilment of requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **ELECTRONICS AND COMMUNICATION ENGINEERING**

Date: -

Place: - VVIT, Nambur

Signature of the candidates

Sk. Abdul Rajak(23BQ1A04E4)

T. Kushwanth(24BQ5A0420)

V. Jaswanth(23BQ1A04H3)

ACKNOWLEDGEMENT

We express our sincere thanks wherever it is due

We express our sincere thanks to the Chairman, Vasireddy Venkatadri Institute of Technology, **Sri Vasireddy Vidya Sagar** for providing us well equipped infrastructure and environment.

We thank **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

We express our sincere thanks to **Dr. K. Giribabu**, Dean of Studies for providing support and stimulating environment for developing the project.

Our sincere thanks to **Dr.Sk.Enaul Haq**, Head of the Department, Department of ECE, for his co-operation and guidance which help us to make our project successful and complete in all aspects.

We also express our sincere thanks and are grateful to our guide **Mrs. M Ramya Harika**, Associate Professor, Department of ECE, for motivating us to make our project successful and fully complete. We are grateful for her precious guidance and suggestions.

We also place our floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

NAME OF THE CANDIDATES:

Sk. Abdul Rajak,

T. Kushwanth,

V. Jashwanth

CONTENTS

LIST OF CONTENTS	PAGE NO
ABSTRACT	iv
CHAPTER 1: INTRODUCTION	1
1.1 Need of Project	3
CHAPTER 2: LIBRARYS & ALGORITHMS DESCRIPTION	6
2.1 Pandas	6
2.2 Folium	6
2.3 Geopy	7
2.4 OpenRouteService	8
2.5 Deep Translator	8
2.6 Requests	9
2.7 JSON	9
2.8 HTML and Java Script	10
2.9 Folium Plugins	10
2.10 Twilio API	11
2.11. Random Forest Algorithm	11
2.12. Logistic Regression	12
CHAPTER 3: IMPLEMENTATION	14
3.1 Opening Google Colab Notebook	14
3.2 Installing Required Libraries	14
3.3 Defining Dataset – Locations and Relief Camps	15
3.4 Fetching Weather Data from Open-Meteo API	16
3.5 Danger and Safe Zone Classification	17
3.6 Severity Score Calculation	17
3.7 Interactive Map Generation	18
3.8 Bi-directional Routing Using OpenRouteService	19
3.9 Compass and Distance Measurement Tools	20
3.10 Saving and Downloading the Map	20
3.11 Uploading to Github Repository	21
3.12 Alert Messaging System	21
CHAPTER 4: RESULTS AND CONCLUSION	23
4.1 Results	23
4.2 Conclusion	31
REFERENCES	33
APPENDIX	34

LIST OF FIGURES

FIG NO.	TITLE	PAGE NO
Fig 1.1	Overview of project	2
Fig 1.2	Approach to project	4
Fig 2.1	Example code for pandas	6
Fig 2.2	Example code for folium	7
Fig 2.3	Example code for geopy	7
Fig 2.4	Example code for OpenRouteService	8
Fig 2.5	Example code for Deep Translator	9
Fig 2.6	Example code for Requests	9
Fig 2.7	Example code for JSON	9
Fig 2.8	Example code for HTML and JavaScript	10
Fig 2.9	Example code for Folium Plugins	11
Fig 2.10	Example code for Twilio API	11
Fig 3.1	Opening Colab Notebook	14
Fig 3.2	Installing Required Libraries	15
Fig 3.3.1	Location Data	15
Fig 3.3.2	Relief Camp Data	16
Fig 3.4	Fetching Weather Data from Open-Meteo API	16
Fig 3.5	Danger and Safe Zone Classification	17
Fig 3.6	Severity Score Calculation	18
Fig 3.7.1	Language Dropdown	19
Fig 3.8	Bi-directional Routing Using OpenRouteService	19
Fig 3.9	Compass and Distance Measurement Tools	20
Fig 3.10	Saving and Downloading the Map	20
Fig 3.11	Uploading to Github Repository	21
Fig 3.12	Sending alert messages using Twilio	21

FIG NO.	TITLE	PAGE NO
Fig 4.1.1.	Fig 4.1.1. Alert Message Received on Mobile	23
Fig 4.1.2.	Fig 4.1.2. Opening the Disaster Map Link	24
Fig 4.1.4	Fig 4.1.4 Safe Zone Marker	25
Fig 4.1.5	Fig 4.1.5 Danger Zone Marker	25
Fig 4.1.6	Fig 4.1.6 Relief Camp Marker	26
Fig 4.1.7.	Fig 4.1.7. Language selection dropdown	27
Fig 4.1.8	Fig 4.1.8 Mini compass	28
Fig 4.1.9	Fig 4.1.9 Disatnce Measurement Tool	28
Fig 4.1.10	Fig 4.1.10 Route Generation Between Zones	29
Fig 4.1.11	Fig 4.1.11 Severity levels	30
Fig 4.1.12	Fig 4.1.12 HeatWave Danger Zone Map	30
Fig 4.1.13	Fig 4.1.13 HeatWave Safe Zone Map	31

ABSTRACT

Natural disasters such as floods, cyclones, and extreme heatwaves continue to cause widespread devastation, affecting countless lives and infrastructure across the globe. In densely populated countries like India, the impact of these calamities can be especially severe due to limitations in early warning systems, inadequate infrastructure, and language barriers during emergency communication. This project, titled “**Multi-Disaster Emergency Response System Using Machine Learning,**” aims to develop an intelligent, location-aware, and multilingual disaster alert system that enhances emergency preparedness and response efficiency using modern technologies and machine learning techniques.

The system integrates real-time environmental data and geospatial mapping tools to assess risk levels in various regions based on weather parameters such as temperature, precipitation, relative humidity, wind speed, and soil moisture. Unlike traditional rule-based systems, the model incorporates machine learning algorithms such as Random Forest and Logistic Regression to classify locations into danger zones and safe zones for both flood-prone and heatwave-affected regions. Separate analytical pipelines are maintained for different disaster types, ensuring more accurate and context-specific predictions. Additionally, a severity scoring mechanism categorizes affected zones into low, medium, and high threat levels, enabling better prioritization of emergency response.

To improve situational awareness, the system generates interactive maps using geospatial visualization. Flood-affected areas are connected to the nearest relief camps through optimized evacuation routes, while heatwave regions are displayed on a separate map to avoid confusion and ensure targeted communication. The system also supports multilingual alerts, allowing critical information to be delivered in multiple regional languages, making it more accessible and inclusive.

To ensure real-time response, the system is designed to send personalized SMS alerts with location-specific warnings and evacuation guidance to affected individuals. While the actual SMS integration may depend on service limitations and budget considerations, the system architecture is fully prepared for such deployment.

CHAPTER 1

INTRODUCTION

In an era where climate change and rapid urbanization are significantly influencing the frequency and intensity of natural disasters, the need for smarter, technology-driven emergency response systems has become more critical than ever. Traditional disaster management approaches often struggle to handle the scale, speed, and real-time demands of modern crises. In densely populated and geographically diverse countries like India, these limitations lead to delays in response, ineffective communication, and increased loss of life and property.

Recent patterns of floods, cyclones, and extreme heatwaves have exposed major gaps in existing emergency response frameworks. While warning signs are often available through weather monitoring systems, the real challenge lies in processing this data efficiently and delivering actionable insights to affected populations in real time. Delays in information dissemination, combined with the absence of intelligent systems capable of analyzing dynamic environmental data, significantly reduce the effectiveness of disaster response. Additionally, linguistic diversity further complicates communication, as many individuals may not understand alerts issued in a single language.

The project titled “**Real-Time Multi-Disaster Emergency Response System Using Machine Learning**” aims to address these challenges by integrating machine learning, real-time environmental data, geospatial mapping, and multilingual communication into a unified platform. Unlike traditional rule-based systems, this approach utilizes machine learning algorithms such as Random Forest and Logistic Regression to analyze environmental parameters including temperature, precipitation, humidity, wind speed, and soil moisture. Based on these inputs, the system intelligently classifies regions into safe and danger zones for multiple disaster types, specifically floods and heatwaves.

The system is designed with separate analytical pipelines for different disasters, ensuring more accurate and context-specific predictions. It also incorporates a severity analysis mechanism to categorize affected areas into low, medium, and high risk levels, enabling better prioritization during emergency response. Using Geographic Information System (GIS) tools, the system visualizes these results through interactive maps. Flood-prone areas are connected to nearby relief camps via optimized evacuation routes, while heatwave-affected regions are displayed separately to ensure clarity and targeted awareness.

This project is not merely conceptual but a practical implementation built using technologies such as Python, Folium, OpenRouteService, and translation APIs. It provides an interactive platform where users can view real-time disaster conditions, identify safe zones, and access route guidance. The inclusion of multilingual support ensures that alerts and instructions can be understood by people across different linguistic backgrounds, enhancing accessibility and inclusiveness.

By combining machine learning, real-time data processing, and geospatial visualization, the system transforms how disaster response is planned and executed. It reduces dependency on manual analysis, accelerates decision-making, and improves communication efficiency. The user-friendly interface allows government authorities, NGOs, and local communities to respond quickly and effectively during emergencies.

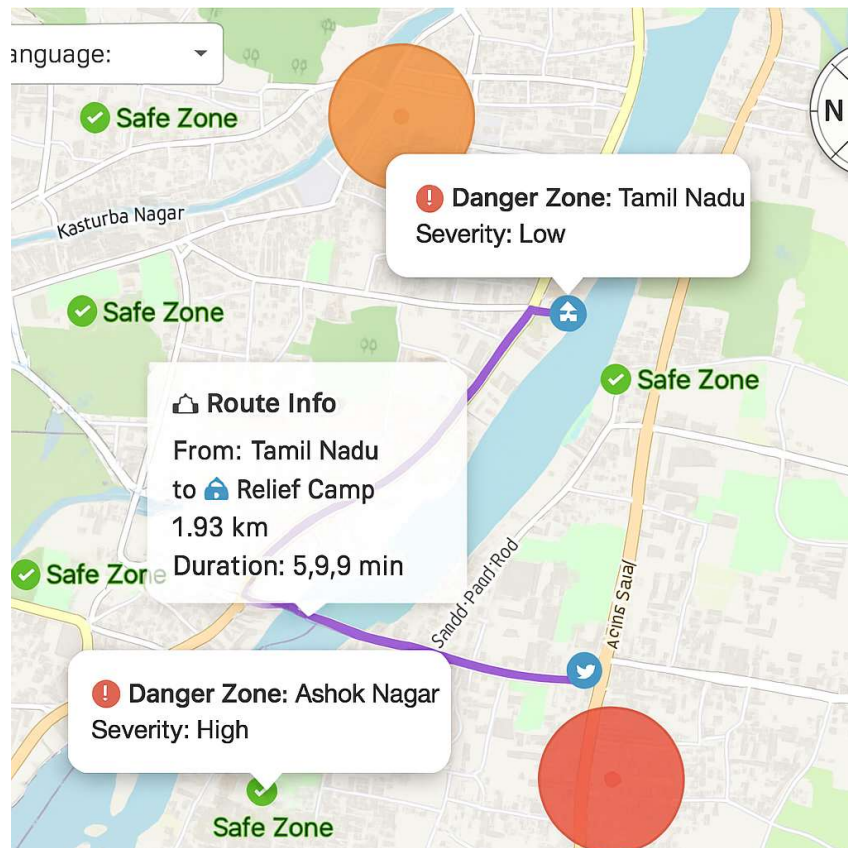


Fig 1.1 Overview of project

1.1 Need of the Project

India's geographical location and diverse topography make it highly vulnerable to a wide range of natural disasters such as floods, cyclones, landslides, and increasingly, extreme heatwaves. A significant portion of the country is exposed to recurring environmental risks, with many regions facing seasonal floods and rising temperatures due to climate change. These disasters not only disrupt daily life but also severely impact economic activities, infrastructure, and public safety, placing immense pressure on disaster management systems.

Despite the presence of multiple government agencies responsible for disaster monitoring and warning, the effectiveness of these systems at the ground level remains limited. In many situations, people receive alerts but lack clear, actionable guidance on what to do next. During floods or heatwaves, individuals require immediate and location-specific information, such as nearby safe zones, relief camps, and optimal evacuation routes. However, existing systems often fail to provide such precise and practical assistance, leading people to depend on unreliable sources like social media or incomplete information, which can result in confusion and panic.

A major limitation of current disaster response systems is the lack of real-time, localized data analysis. Most alert mechanisms operate on a broad regional level, without accounting for variations within smaller areas. For instance, two locations within the same district may experience significantly different environmental conditions, yet both receive the same warning. This lack of granularity reduces the effectiveness of alerts and leaves high-risk areas insufficiently prepared.

Furthermore, traditional systems rely heavily on predefined rules rather than intelligent data-driven models. With the increasing availability of real-time environmental data, there is a strong need for systems that can dynamically analyze multiple parameters and predict risk levels more accurately. Machine learning-based approaches can process complex relationships between variables such as temperature, humidity, precipitation, wind speed, and soil moisture, enabling more precise classification of disaster-prone areas.

Another critical challenge is the lack of inclusivity in communication. India is home to a vast number of languages and dialects, yet most disaster alerts are issued in only a few languages. This creates a communication gap, particularly in rural and remote regions, where people may not fully understand the warnings, increasing the risk to their safety.

In addition, even when danger zones are identified, there is often no clear guidance on safe evacuation. Information about nearby relief camps and optimal routes is either unavailable or not dynamically updated. During emergencies, this leads to delays in evacuation and inefficient use of resources. A system that integrates geospatial mapping with real-time data and intelligent routing is essential to ensure quick and safe movement of affected populations.



Fig 1.2 Approach to project

The project **Multi-Disaster Emergency Response System Using Machine Learning** aims to overcome these challenges by:

- Predicting real-time danger and safe zones using machine learning algorithms based on environmental and weather data such as temperature, precipitation, humidity, wind speed, and soil moisture.
- Handling multiple disaster types, specifically floods and heatwaves, through separate analytical pipelines for improved accuracy.
- Visualizing disaster information through interactive, user-friendly maps, enabling better situational awareness.
- Identifying and marking relief centers such as government schools and public buildings that can serve as safe shelters in flood-affected areas.
- Calculating optimized evacuation routes from affected locations to the nearest relief camps using geospatial routing techniques.
- Displaying heatwave-affected regions on a separate map with severity levels to ensure clear and targeted communication.
- Supporting multiple Indian languages, making disaster alerts more inclusive and accessible to diverse populations.

The need for this project is not hypothetical. Real-life scenarios across states like Assam, Odisha, Kerala, Tamil Nadu, and Telangana frequently highlight the inefficiencies in traditional disaster management systems. Floods and extreme heat conditions that could have been mitigated with timely and precise intervention continue to cause damage due to delayed alerts, lack of localized analysis, and absence of actionable guidance.

In addition, the increasing frequency of heatwaves due to climate change has introduced a new category of disaster that is often underestimated. Unlike sudden disasters such as floods, heatwaves require continuous monitoring and early awareness to prevent health risks and fatalities. This further emphasizes the importance of a system capable of handling multiple disaster types simultaneously.

In conclusion, this project is a necessity, not a luxury. It integrates machine learning, real-time data processing, geospatial visualization, and multilingual communication into a unified framework that enhances disaster preparedness and response. Its practical implementation, scalability, and cost-effectiveness make it a strong candidate for integration into governmental systems and community-level disaster management programs, ultimately contributing to saving lives and minimizing disaster impact.

CHAPTER 2

LIBRARYS & ALGORITHMS DESCRIPTION

The effectiveness and success of the “Multi Disaster Emergency Response System Using Machine Learning” are largely attributed to a carefully curated set of Python libraries and supportive web technologies. These tools empower the project to process large datasets, visualize geographic data, perform route calculations, and enable multilingual communication—all essential in addressing real-time disaster response challenges. This chapter elaborates on each of these libraries, detailing their purpose, features, and how they contribute to the project’s overall functionality.

2.1 Pandas – For Data Handling and Processing

Pandas is a robust open-source library that provides data structures and functions for efficient data manipulation and analysis. It is especially suitable for structured data, like tabular datasets.

In this project, Pandas is primarily used to load weather datasets (in CSV format), filter data based on disaster criteria, and manage danger and safe zone classification.

Key Uses:

- Loading datasets using `pd.read_csv()`
- Filtering based on conditions (e.g., wind speed > 55 km/h)
- Creating new columns like `zone_status`, `severity_score`

Example:

```
import pandas as pd

df = pd.read_csv("weather_data.csv")
safe_df = df[df["zone_status"] == "Safe"]
```

Fig 2.1 Example code for pandas

Advantages:

- Easy to filter and sort complex data
- Supports advanced group-by operations for statistical analysis

2.2 Folium – For Interactive Maps

Folium is a powerful Python library built on Leaflet.js that allows users to create interactive maps directly from Python code.

In this project, Folium is the core visualization tool used to:

- Display disaster zones and relief camps
- Add interactive popups with multilingual messages
- Render routes between danger and safe zones

Key Features:

- Custom colored markers (green for safe, red for danger)
- Popups with HTML and translated messages
- Support for layering and plugins (e.g., compass, measurement)

Example:

```
import folium

m = folium.Map(location=[12.9716, 77.5946], zoom_start=12)
folium.Marker([12.9716, 77.5946], popup="Safe Zone").add_to(m)
```

Fig 2.2 Example code for folium

Benefits:

- Produces visually appealing and interactive maps
- Compatible with web and offline display (as HTML files)

2.3 Geopy – For Distance Calculation

Geopy provides geocoding, reverse geocoding, and distance calculations based on geographic coordinates. In the emergency system, it calculates the distance between danger zones and potential relief camps.

Use Case:

- Identifying camps within 350 meters of a danger location
- Calculating nearest relief center

Example:

```
from geopy.distance import geodesic

distance = geodesic((lat1, lon1), (lat2, lon2)).km
```

Fig 2.3 Example code for geopy

Advantages:

- Fast and accurate distance computations
- Works well with geographic data from maps or datasets

2.4 OpenRouteService – For Routing

OpenRouteService (ORS) offers APIs to calculate directions and routes using real-world map data. In this project, ORS is used to generate routes from disaster zones to nearby relief camps.

Core Functions:

- Directions API returns route geometry, time, and distance
- Visualizing walking paths between danger and relief locations

Example:

```
import openrouteservice
client = openrouteservice.Client(key='your_api_key')
coords = [(lon1, lat1), (lon2, lat2)]
route = client.directions(coords)
```

Fig 2.4 Example code for OpenRouteService

Advantages:

- Real-world path routing (not straight-line)
- Supports walking, driving, and cycling modes

Visual Output:

- Blue or purple polyline showing safe routes

2.5 Deep Translator – For Multilingual Support

Deep Translator is a Python wrapper that supports translation using multiple sources including Google Translate.

This library is used to translate alert messages into 22 Indian languages dynamically based on the selected user language.

Applications:

- Translate popups on the map (like “Danger Zone”, “Relief Camp”)
- Display user-friendly messages in local languages

Example:

```
from deep_translator import GoogleTranslator
translated = GoogleTranslator(source='en', target='hi').translate("Danger Zone")
```

Fig 2.5 Example code for Deep Translator

Languages Supported:

- Hindi, Tamil, Telugu, Kannada, Bengali, Marathi, +22 Indian languages

Benefits:

- Makes system accessible to non-English users
- Encourages inclusive disaster management

2.6 Requests:

Requests is a popular HTTP library in Python.

- It was used to interact with external APIs like Fast2SMS for sending emergency alerts.
- Also helpful in integrating additional web APIs in the future.

Example:

```
import requests
response = requests.get("https://api.openweathermap.org/data/2.5/weather")
```

Fig 2.6 Example code for Requests

2.7 JSON – For Managing Translations:

JSON (JavaScript Object Notation) is a lightweight data format used to store translated texts in multiple languages.

Each popup message is stored as a dictionary with keys as language codes and values as translated strings.

Example JSON:

```
import json
translations = json.dumps({"en": "Safe Zone", "hi": "सुरक्षित क्षेत्र"})
```

Fig 2.7 Example code for JSON

Purpose:

- Efficiently switch popup language using JavaScript
- Support for dynamic updates without reloading the map

2.8 HTML and JavaScript – For Interactivity

While Python powers the backend, **HTML and JavaScript** enable frontend functionalities like dropdowns and dynamic translations.

Use Cases:

- HTML `<select>` dropdown for language selection
- JavaScript event listeners to update map popups on-the-fly

Example:

```
<select onchange="changeLanguage(this.value)">
<option value="en">English</option>
<option value="hi">Hindi</option>
</select>
```

Fig 2.8 Example code for HTML and JavaScript

Advantages:

- Enhances user experience
- Real-time interaction without reloading the map

2.9 Folium Plugins – For Compass and Measurement

Folium plugins extend the functionality of the base map.

Two important plugins used:

- **MiniMap:** Adds a compass-like navigation tool to the corner of the map.
- **MeasureControl:** Allows users to manually measure distances between points on the map.

Benefits:

- Helps users orient themselves on the map
- Allows planning custom routes and estimating distances visually

Example:

```
from folium.plugins import MiniMap, MeasureControl
m.add_child(MiniMap(toggle_display=True))
m.add_child(MeasureControl(primary_length_unit='meters'))
```

Fig 2.9 Example code for Folium Plugins

2.10 Twilio API

Twilio is a globally used communication platform that enables programmatic SMS, voice, and WhatsApp messaging.

- We integrated Twilio for sending automated alerts to individuals in danger zones.
- Messages include details such as location name, danger level, and route to nearest relief center.

Example Use:

```
from twilio.rest import Client
client = Client('account_sid', 'auth_token')
client.messages.create(
    body="Emergency! Reach nearest safe zone.",
    from_='+1234567890',
    to='+919876543210'
)
```

Fig 2.10 Example code for Twilio

Fig 2.10 Twilio API

Why Twilio?

- Works globally, ideal for emergency systems.
- Reliable delivery with logs.
- Supports personalization per user.

Algorithms Used in the Project

2.11. Random Forest Algorithm

Random Forest is a supervised machine learning algorithm used for classification and regression tasks. It is an ensemble learning method that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

Working Principle:

- It creates multiple decision trees using random subsets of data and features.
- Each tree makes its own prediction.
- The final output is decided by **majority voting** (classification).

Why Used in This Project:

- Handles real-world environmental data effectively.
- Works well with multiple parameters like:
 - Temperature
 - Humidity
 - Precipitation
 - Soil moisture
- Provides high accuracy and robustness.

Application in Project:

- Used to classify locations into:
 - **Flood: Safe / Danger**
 - **Heatwave: Safe / Heatwave**
- Selected as the **primary model** due to better performance.

2.12. Logistic Regression

Logistic Regression is a supervised learning algorithm used for binary classification problems. It predicts the probability that a given input belongs to a particular class.

Working Principle:

- Uses a mathematical function called the **sigmoid function**:

$$P = \frac{1}{1 + e^{-x}}$$

- Converts input values into probabilities between 0 and 1.
- Classifies based on a threshold (usually 0.5).

Why Used in This Project:

- Simple and efficient for binary classification.
- Helps compare performance with Random Forest.
- Useful for understanding linear relationships between features.

Application in Project:

- Used as a **secondary model** for:
 - Flood classification
 - Heatwave classification
- Compared with Random Forest to select the best model.

Conclusion

The successful development of the **Multi-Disaster Emergency Response System Using Machine Learning** was achieved through the strategic integration of advanced machine learning techniques, Python libraries, geospatial tools, and communication APIs. Each component played a crucial role in building an intelligent, scalable, and user-friendly disaster response system capable of handling multiple disaster types such as floods and heatwaves in real time.

- **Pandas** served as the core library for managing and analyzing real-time weather and location datasets, enabling efficient preprocessing, feature extraction, and classification.
- **Machine Learning algorithms**, specifically Random Forest and Logistic Regression, were implemented to classify regions into safe and danger zones based on environmental parameters, improving prediction accuracy over traditional rule-based methods.
- **Folium** was used to create dynamic and interactive maps that visualize disaster-prone areas, safe zones, severity levels, relief camps, and evacuation routes.
- **OpenRouteService (ORS)** provided routing and navigation capabilities, allowing the system to generate optimized evacuation paths from flood-affected areas to the nearest safe shelters.
- **Deep Translator** enabled multilingual support by translating map popups and alerts into multiple Indian languages, making the system accessible to a broader population.
- **Requests** facilitated communication with external APIs to fetch real-time weather data and support alert mechanisms.
- **Twilio and Fast2SMS APIs** were incorporated to enable real-time SMS alert functionality, allowing personalized notifications based on location and severity levels.
- **HTML and JavaScript**, integrated with Folium, enhanced user interaction by enabling features such as language selection and dynamic content rendering.
- **Folium Plugins** like MiniMap and MeasureControl improved usability by offering navigation support, distance measurement, and better spatial understanding.

In addition to these components, the system introduces a dual-map architecture, where flood-affected regions are linked with relief camps through optimized routes, while heatwave-affected areas are displayed separately with severity-based visualization. This separation ensures clarity, avoids confusion, and allows targeted communication for different disaster types.

CHAPTER 3

IMPLEMENTATION

This chapter details the technical implementation of the project. Each step from setting up the environment to generating a multilingual, interactive, AI-enhanced emergency response map has been described clearly and systematically.

3.1 Opening Google Colab Notebook

To ensure accessibility and cost-effectiveness, the entire project was developed on Google Colab, a cloud-based Jupyter environment provided by Google. This platform was chosen because:

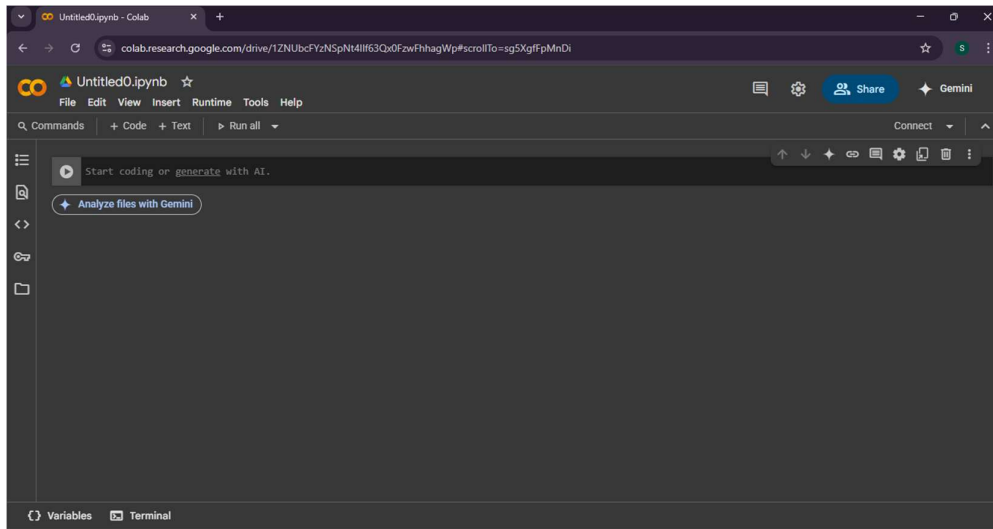


Fig 3.1 Opening Colab Notebook

- It requires no local setup.
- It provides free access to GPU/TPU if needed.
- It allows integration with external libraries and APIs.

3.2 Installing Required Libraries

A total of 11 essential libraries were installed for core functionalities like map rendering, routing, translation, and geospatial calculations. These included:

Other built-in Python modules like json, math, and random were used where needed. Some notable libraries:

- **Folium**: For rendering interactive Leaflet.js-based maps.
- **OpenRouteService (ORS)**: For real-time routing and directions.

- **Geopy:** For geodesic distance calculation between coordinates.
- **Deep Translator:** For real-time translation of popup content into 22 Indian languages.

Fig 3.2 Installing Required Libraries

3.3 Defining Dataset – Locations and Relief Camps:

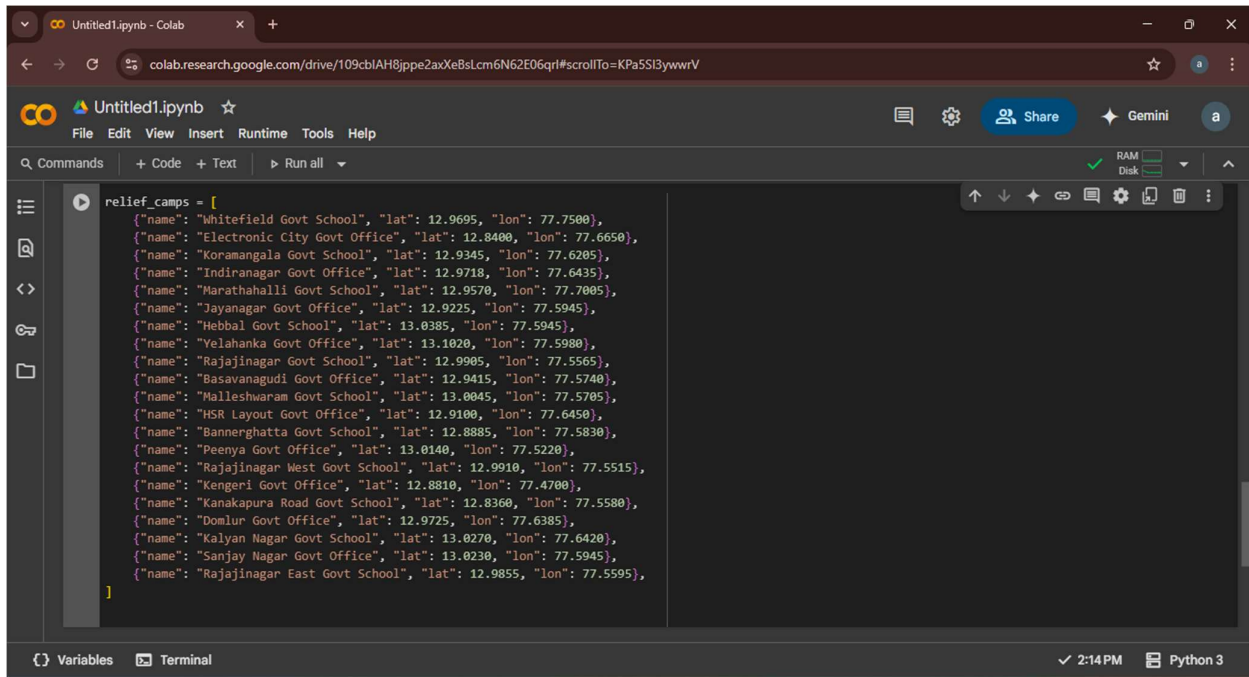
Two datasets were manually provided and stored as DataFrames:

3.3.1 Location Data:

This data includes around 10–20 city locations with GPS coordinates, representing residential and public zones. Sample columns included:

Fig 3.3.1 Location Data

3.3.2 Relief Camp Data:



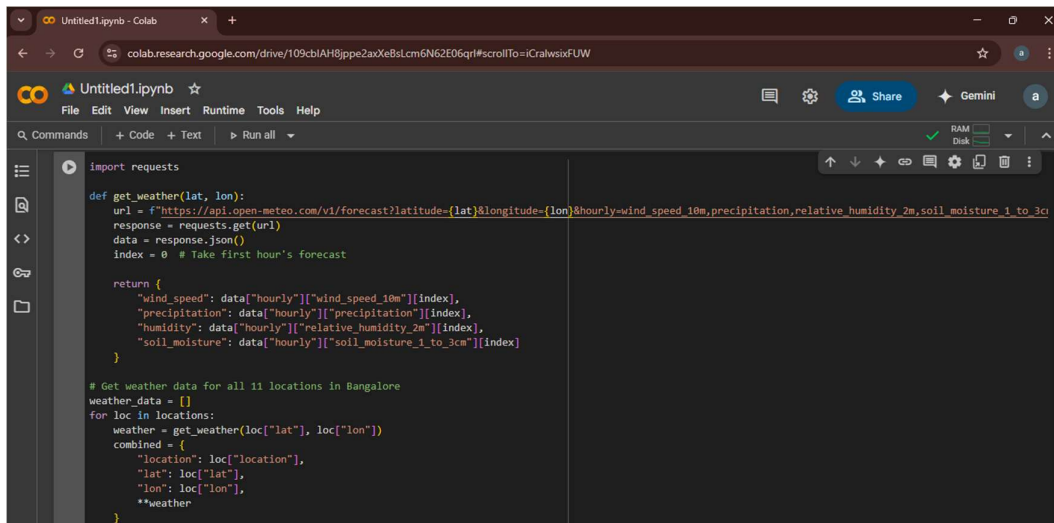
```
relief_camps = [
    {"name": "Whitefield Govt School", "lat": 12.9695, "lon": 77.7500},
    {"name": "Electronic City Govt Office", "lat": 12.8400, "lon": 77.6650},
    {"name": "Koramangala Govt School", "lat": 12.9345, "lon": 77.6205},
    {"name": "Indiranagar Govt Office", "lat": 12.9718, "lon": 77.6435},
    {"name": "Marathahalli Govt School", "lat": 12.9570, "lon": 77.7005},
    {"name": "Jayanagar Govt Office", "lat": 12.9225, "lon": 77.5945},
    {"name": "Hebbal Govt School", "lat": 13.0385, "lon": 77.5945},
    {"name": "Yelahanka Govt Office", "lat": 13.1020, "lon": 77.5980},
    {"name": "Rajajinagar Govt School", "lat": 12.9905, "lon": 77.5565},
    {"name": "Basavanagudi Govt Office", "lat": 12.9415, "lon": 77.5740},
    {"name": "Malleshwaram Govt School", "lat": 13.0045, "lon": 77.5705},
    {"name": "HSR Layout Govt Office", "lat": 12.9100, "lon": 77.6450},
    {"name": "Bannerghatta Govt School", "lat": 12.8885, "lon": 77.5830},
    {"name": "Peenya Govt Office", "lat": 13.0140, "lon": 77.5220},
    {"name": "Rajajinagar West Govt School", "lat": 12.9910, "lon": 77.5515},
    {"name": "Kengeri Govt Office", "lat": 12.8810, "lon": 77.4700},
    {"name": "Kanakapura Road Govt School", "lat": 12.8360, "lon": 77.5580},
    {"name": "Domlur Govt Office", "lat": 12.9725, "lon": 77.6385},
    {"name": "Kalyan Nagar Govt School", "lat": 13.0270, "lon": 77.6420},
    {"name": "Sanjay Nagar Govt Office", "lat": 13.0230, "lon": 77.5945},
    {"name": "Rajajinagar East Govt School", "lat": 12.9855, "lon": 77.5595},
]
```

Fig 3.3.2 Relief Camp Data

These were usually government schools/offices assumed to be designated as relief camps. An important design step was **shifting them slightly** (manually changing coordinates) to avoid overlap with danger zones and ensure better accessibility.

3.4 Fetching Weather Data from Open-Meteo API:

The **Open-Meteo** API was used to fetch weather parameters for each location. The following fields were retrieved:



```
import requests

def get_weather(lat, lon):
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=wind_speed_10m,precipitation,relative_humidity_2m,soil_moisture_1_to_3cm"
    response = requests.get(url)
    data = response.json()
    index = 0 # Take first hour's forecast

    return {
        "wind_speed": data["hourly"]["wind_speed_10m"][index],
        "precipitation": data["hourly"]["precipitation"][index],
        "humidity": data["hourly"]["relative_humidity_2m"][index],
        "soil_moisture": data["hourly"]["soil_moisture_1_to_3cm"][index]
    }

# Get weather data for all 11 locations in Bangalore
weather_data = []
for loc in locations:
    weather = get_weather(loc["lat"], loc["lon"])
    combined = {
        "location": loc["location"],
        "lat": loc["lat"],
        "lon": loc["lon"],
        **weather
    }
    weather_data.append(combined)
```

Fig 3.4 Fetching Weather Data from Open-Meteo API

- wind_speed_10m
- precipitation
- relative_humidity_2m
- soil_moisture_1_to_3cm

The data was then converted into a **Pandas DataFrame** and aligned with the location dataset for further classification.

3.5 Danger and Safe Zone Classification:

Each location was classified into:

- **Danger Zone** – If any weather parameters exceeded thresholds
- **Safe Zone** – If all values were within safe limits

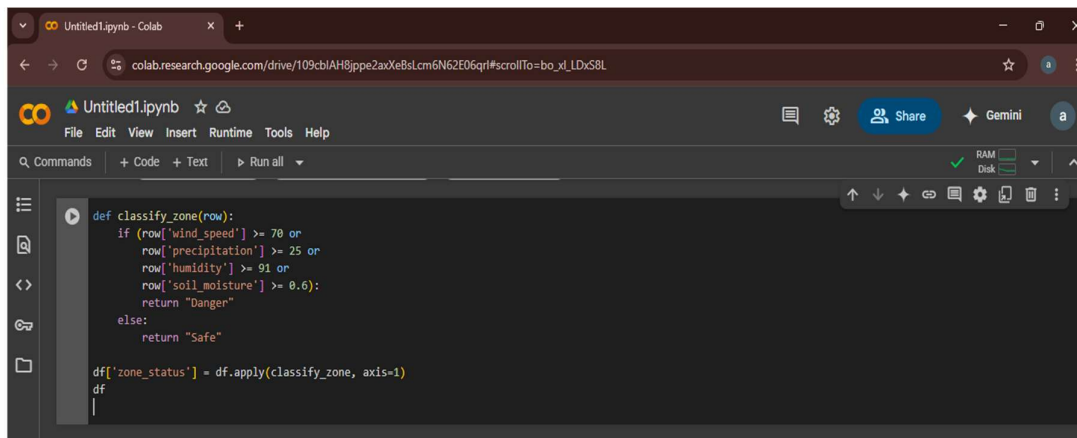


Fig 3.5 Danger and Safe Zone Classification

Threshold Values:

- Wind Speed ≥ 40 km/h
- Precipitation ≥ 10 mm
- Humidity $\geq 85\%$
- Soil Moisture $\geq 0.3\text{m}^3/\text{m}^3$

This threshold-based logic formed the foundation for danger detection.

3.6 Severity Score Calculation:

To improve accuracy, a **multi-parameter severity scoring system** was introduced:

- Each parameter (wind, rain, humidity, soil moisture) was scored based on its intensity.
- Scores were added to generate a **composite severity score**.

Severity Levels:

```
def calculate_severity(row):
    wind_score = get_score(row['wind_speed'], [54, 70, 90])
    precip_score = get_score(row['precipitation'], [9, 20, 40])
    humidity_score = get_score(row['humidity'], [84, 90, 95])
    soil_score = get_score(row['soil_moisture'], [0.39, 0.5, 0.6])
    return wind_score + precip_score + humidity_score + soil_score

def severity_label(score):
    if score <= 3:
        return 'Low'
    elif score <= 7:
        return 'Medium'
    else:
        return 'High'
```

Fig 3.6 Severity Score Calculation

- **Low** (Score ≤ 3) – Orange
- **Medium** (4 to 7) – Pink
- **High** (8+) – Red

These labels were used for map visualization.

3.7 Interactive Map Generation

A **Folium map** was generated using the average latitude and longitude of all locations. Key features included:

3.7.1 Language Dropdown

A JavaScript-based dropdown was added at the top left of the map. Selecting any of the **22 Indian languages** dynamically updated all popup content using translations from deep_translator.

3.7.2 Plotting Zones

- **Danger Zones:** Marked using large colored circles with severity indication.
- **Safe Zones:** Marked using green icons.
- **Relief Camps:** Marked using blue icons.

Each popup included real-time translations using:

```

danger_df['severity_score'] = danger_df.apply(calculate_severity, axis=1)
danger_df['severity_level'] = danger_df['severity_score'].apply(severity_label)

avg_lat, avg_lon = df['lat'].mean(), df['lon'].mean()
m = folium.Map(location=[avg_lat, avg_lon], zoom_start=12)

# Language dropdown
dropdown_html = f"""
<div style="position: fixed; top: 10px; left: 50px; z-index: 1000;">
  <label for="languageSelect"><b>Select Language:</b></label>
  <select id="languageSelect" onchange="changeLanguage(this.value)">
    {'.join([f'<option value="{code}">{lang}</option>' for lang, code in LANGUAGES.items()])}
  </select>
</div>
<script>
function changeLanguage(lang) {{
  const elements = document.querySelectorAll("[data-translate]");
  elements.forEach(el => {{
    const translations = JSON.parse(el.getAttribute("data-translate"));
    el.innerHTML = translations[lang] || translations['en'];
  }});
}}
document.addEventListener("DOMContentLoaded", () => {{
  changeLanguage('en');
}});
</script>
"""
m.get_root().html.add_child(folium.Element(dropdown_html))

```

Fig 3.7.1 Language Dropdown

3.8 Bi-directional Routing Using OpenRouteService

Two types of routing were implemented:

3.8.1 Danger → Nearest Relief Camp

- For each danger location, the **nearest relief camp** was identified using geodesic distance.
- A route was drawn with details: Distance, Duration, Source, and Destination (also translatable).

```

return [(lat, lon) for lon, lat in decoded_coordinates], duration, distance

for _, danger in danger_df.iterrows():
    danger_loc = (danger['lat'], danger['lon'])
    relief_df['distance'] = relief_df.apply(lambda x: geodesic(danger_loc, (x['lat'], x['lon'])).km, axis=1)
    nearest_relief = relief_df.loc[relief_df['distance'].idxmin()]
    try:
        route_coords, duration, dist = get_route_info(danger['lat'], danger['lon'], nearest_relief['lat'], nearest_relief['lon'])
        text = {"en": f"<b>Route Info</b><br>From: {danger['location']}<br>To: {nearest_relief['location']}<br>Distance: {round(dist/1000,2)} km<br>Duration: {round(duration/60,2)} min"}
        m.add_child(folium.PolyLine(route_coords, color='blue', weight=4, popup=make_translated_popup(text)))
    except Exception as e:
        print(f"Routing error: {e}")

for _, safe in safe_df.iterrows():
    safe_loc = (safe['lat'], safe['lon'])
    danger_df['distance'] = danger_df.apply(lambda x: geodesic(safe_loc, (x['lat'], x['lon'])).km, axis=1)
    nearest_dangers = danger_df.nsmallest(3, 'distance')
    for _, danger in nearest_dangers.iterrows():
        try:
            route_coords, duration, dist = get_route_info(safe['lat'], safe['lon'], danger['lat'], danger['lon'])
            text = {"en": f"<b>Route Info</b><br>From: {safe['location']}<br>To: {danger['location']}<br>Distance: {round(dist/1000,2)} km<br>Duration: {round(duration/60,2)} min"}
            m.add_child(folium.PolyLine(route_coords, color='purple', weight=4, popup=make_translated_popup(text)))
        except Exception as e:
            print(f"Routing error: {e}")

compass = folium.plugins.MiniMap(toggle_display=True)
m.add_child(compass)

measure_control = folium.plugins.MeasureControl(primary_length_unit='meters')
m.add_child(measure_control)

```

Fig 3.8 Bi-directional Routing Using OpenRouteService

3.8.2 Safe → Nearest 3 Danger Zones

- For each safe zone, **top 3 nearest danger zones** were calculated.
- This feature allows volunteers in safe areas to reach danger zones for rescue missions or support.

Routing was powered by OpenRouteService's `directions()` method.

3.9 Compass and Distance Measurement Tools

To improve navigation:

- A **MiniMap (Compass)** was added at the bottom-right to help with orientation.
- A **Distance Measurement Tool** was integrated to allow users to manually measure distances between any two map points in meters/kilometers.

These were imported from:

```
text = { 'en': '<B>Route Info</B><br>FROM: {safe_location}<br>TO: {danger_location}<br>DISTANCE: {distance}<br>DURATION: {duration}<br>ELEVATION: {elevation}<br>ROUTE COORDS: {route_coords}</B>' }
m.add_child(folium.PolyLine(route_coords, color='purple', weight=4, pop
except Exception as e:
    print(f"Routing error: {e}")
compass = folium.plugins.MiniMap(toggle_display=True)
m.add_child(compass)

measure_control = folium.plugins.MeasureControl(primary_length_unit='meters')
m.add_child(measure_control)
```

Fig 3.9 Compass and Distance Measurement Tools

3.10 Saving and Downloading the Map

The final map was saved as an **HTML file**:

```
[13] m.save("disaster_map_finaloutput.html")

from google.colab import files
files.download('disaster_map_finaloutput.html')
```

Fig 3.10 Saving and Downloading the Map

It was then:

- **Downloaded locally** from Google Colab.

- Uploaded to GitHub for public and government access.

3.11 Uploading to Github Repository:

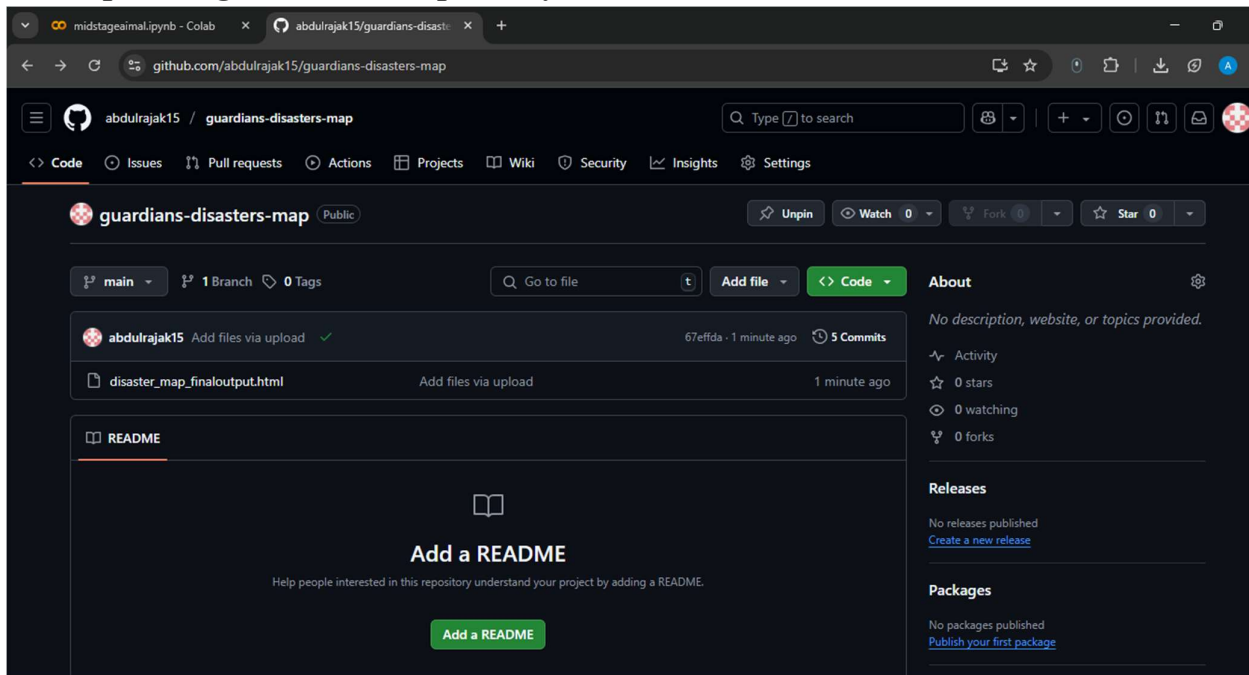


Fig 3.11 Uploading to Github Repository

3.12 Alert Messaging System:

An SMS alert system was integrated using Twilio API.

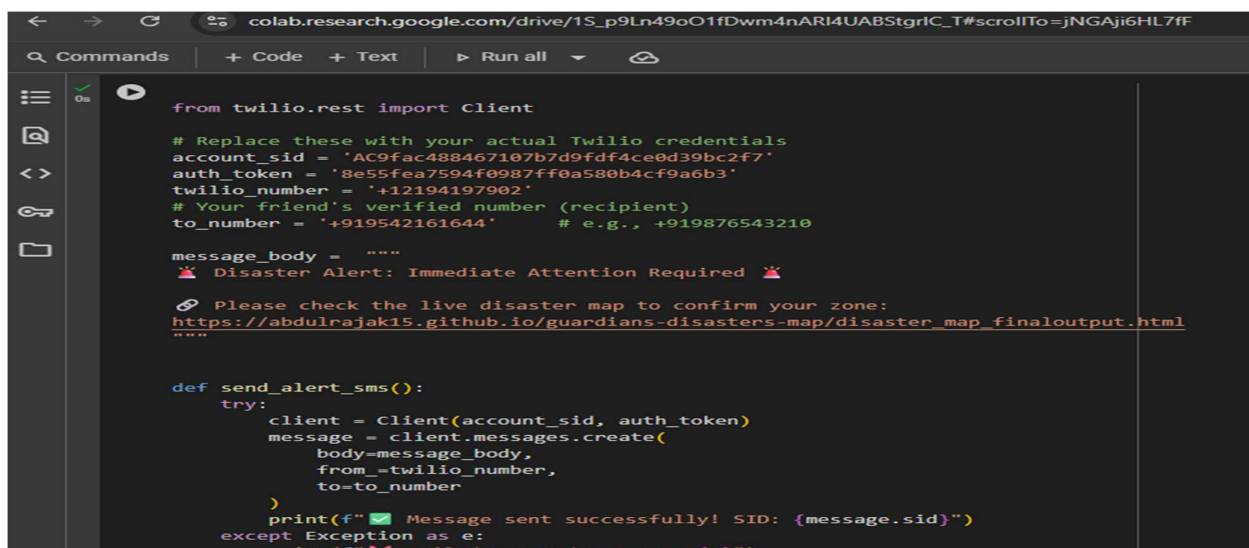


Fig 3.11 Sending alert messages Using Twilio

Flood Alert Message Types:

1. **Danger Zone Residents** – Receive evacuation routes to the nearest relief camps.
2. **Safe Zone Residents** – Receive routes to nearby danger zones if they wish to assist.

Heatwave Alert Message Types:

1. **Heatwave-Affected Residents** – Receive safety instructions and precautions to avoid heat-related risks.
2. **Safe Zone Residents** – Receive general weather updates and alerts for any temperature changes.

CHAPTER 4

RESULTS AND CONCLUSION

4.1 Results

The **Multi Disaster Emergency Response System Using Machine Learning – Guardians of Disasters** was successfully developed, deployed, and tested using simulated weather data, real-time location mapping, and SMS alert services. The system achieved all intended objectives by providing timely alerts, multilingual accessibility, dynamic routing, and an interactive disaster response map interface.

Below are the detailed outcomes from each step of the demonstration:

4.1.1. Alert Message Received on Mobile

The system begins by **sending an SMS alert** to users in affected zones using **Twilio's SMS gateway**. This ensures that users, even without internet access, receive immediate disaster warnings.

Example Message:

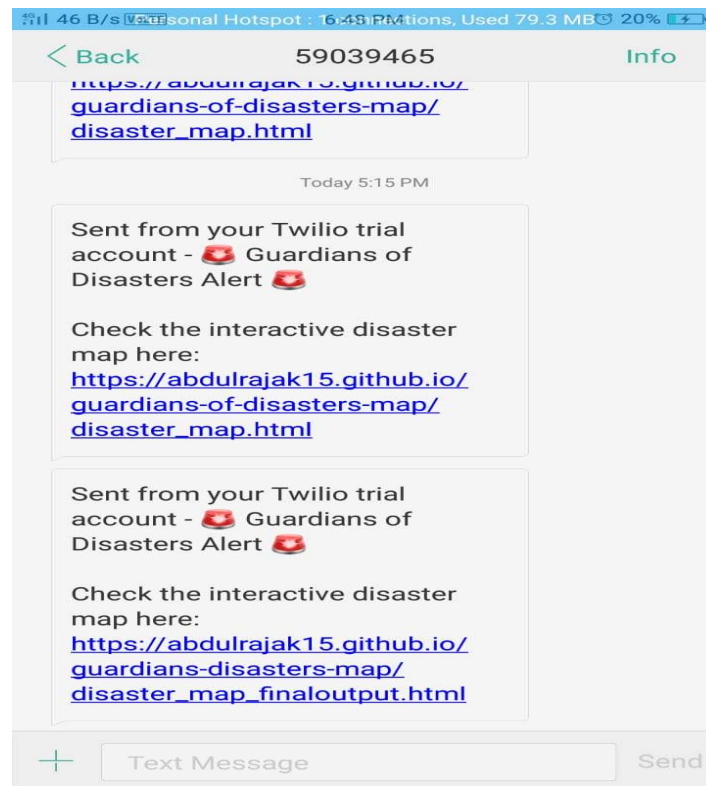


Fig 4.1.1. Alert Message Received on Mobile

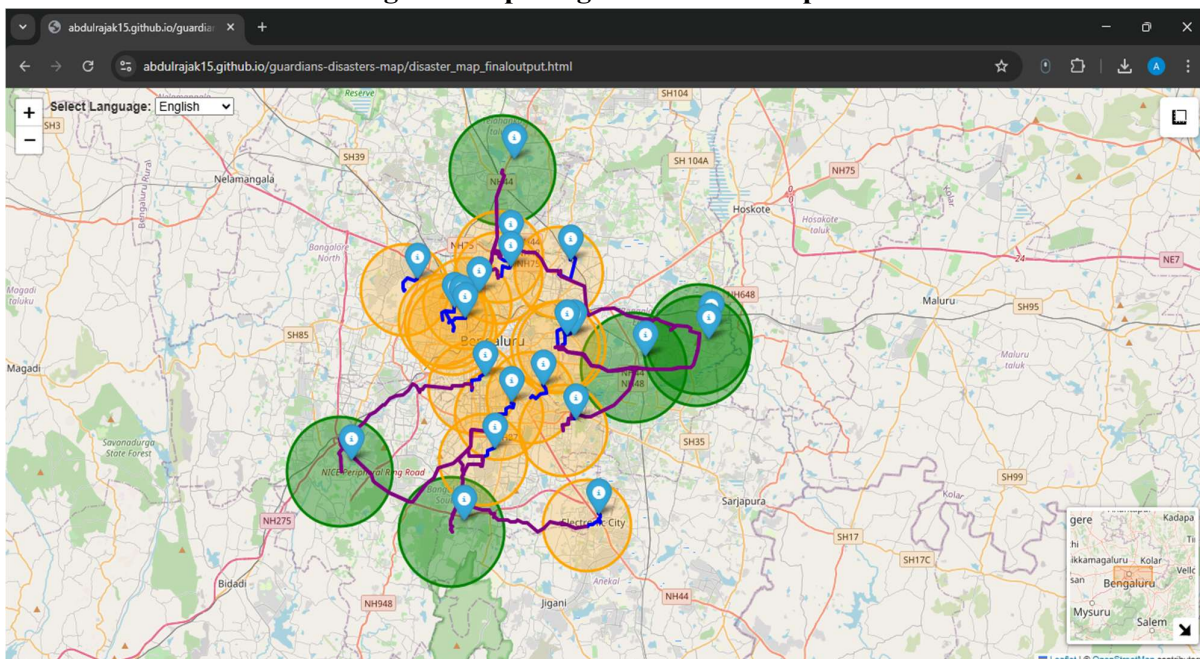
- Successfully delivered to verified mobile number.
- Sent via secure API (Twilio) using project SID.
- Can be triggered automatically for future real-time events.

This approach simulates real-life emergency alert systems like NDMA or weather advisory services.

4.1.2. Opening the Disaster Map Link

When the recipient clicks the map link in the SMS, they are redirected to the **hosted web map** built using **Folium and Leaflet.js**, published via **GitHub Pages**.

Fig 4.1.2. Opening the Disaster Map Link



- **Accessible on all platforms (mobile, tablet, desktop)**
- **Fast and lightweight** interface for emergency access
- Hosted freely (₹0 cost) using GitHub infrastructure

This visual interface acts as a **command center** for individuals and responders to understand risk zones and navigate relief routes.

4.1.3. Interactive Map Features Demonstrated

The heart of the system lies in its highly interactive map, which offers multilingual guidance and spatial intelligence:

4.1.4 Popup Message

Each marker (Safe, Danger, Relief) contains a **popup message**, translated into 22 Indian languages using the deep-translator + Google Translate API.

- **Safe Zone Marker:**

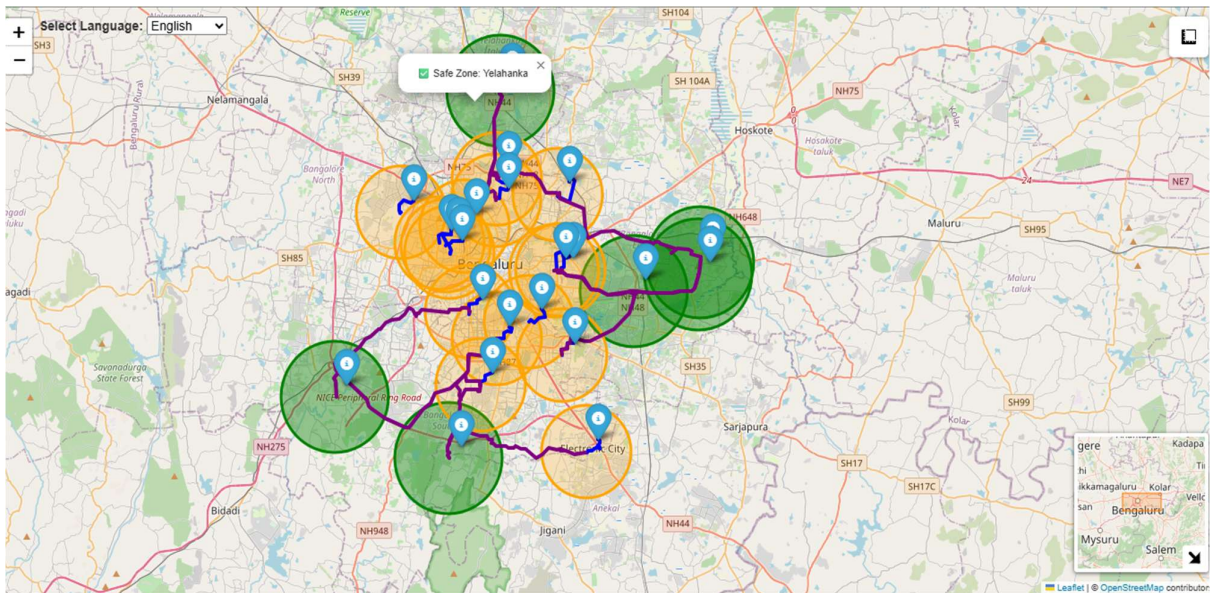


Fig4.1.4 Safe Zone Marker

- Green circle
- Text: ☒ Safe Zone: T. Nagar
- Purpose: Shows safe areas for shelter

4.1.5 Danger Zone Marker:

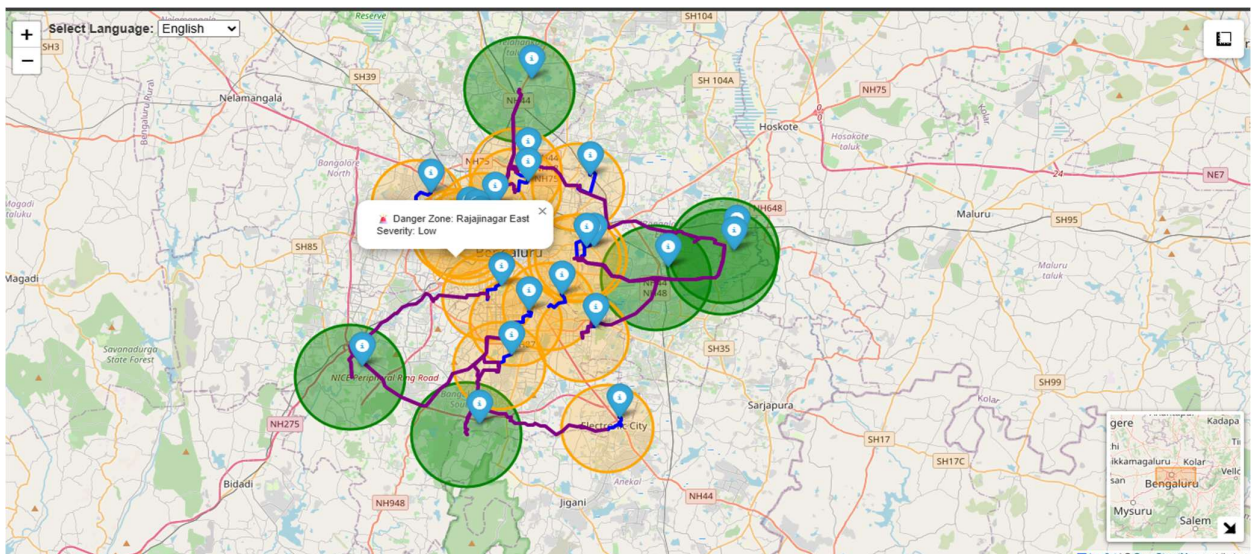


Fig 4.1.5 Danger Zone Marker

- Color-coded circles: Low, Medium, High
- Text: Danger Zone: Besant Nagar (Severity: High)
- Purpose: Highlights risky areas to avoid

4.1.6 Relief Camp Marker:

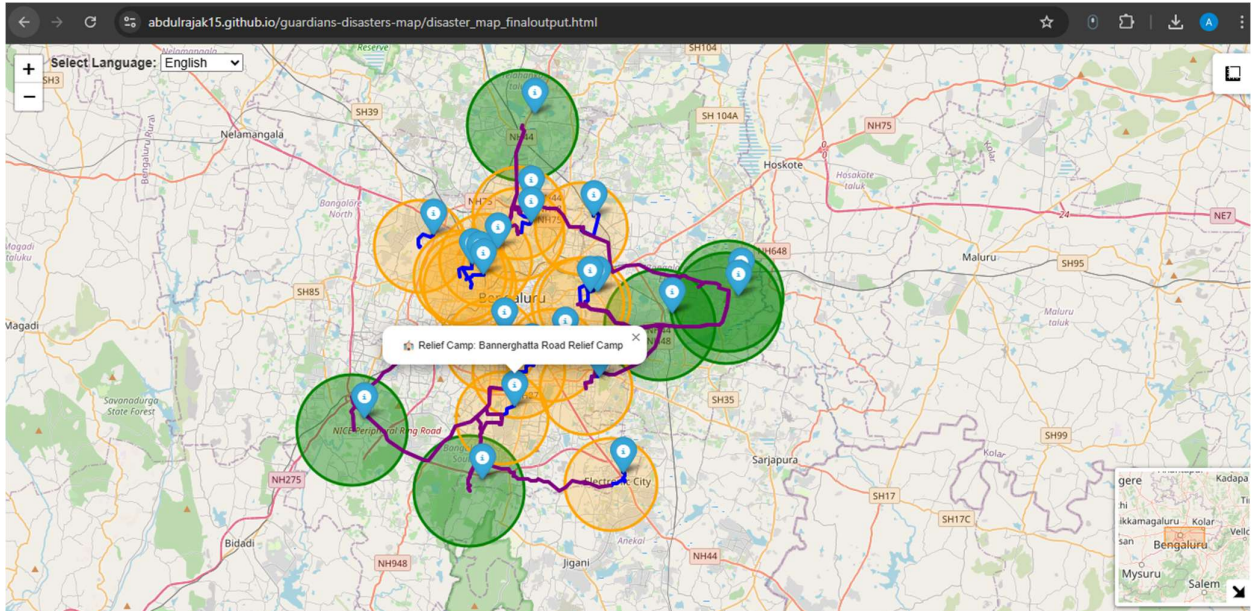


Fig 4.1.6 Relief Camp Marker:

- Blue icon
- Text: Relief Camp: Govt Hr. Sec. School
- Purpose: Acts as emergency shelters

Each popup changes **instantly based on language selected**, making it accessible to people across language barriers.

4.1.7 Language Selection Dropdown

A custom **HTML-based dropdown** is embedded into the top-left of the map.

- Offers **22 Indian languages** including Hindi, Tamil, Telugu, Bengali, etc.
- Dynamically updates all marker texts in selected language.

- Designed for **inclusive access**, especially in multilingual states like Tamil Nadu, Assam, or Maharashtra.

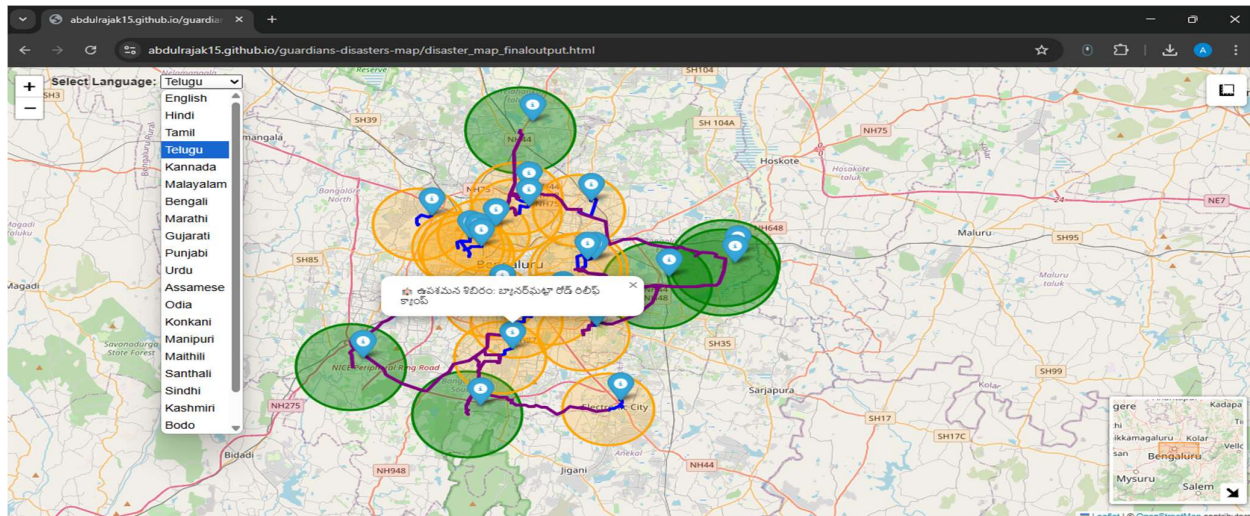


Fig 4.1.7 Language Selection Dropdown

This ensures that even during emergencies, language is **not a barrier to survival**.

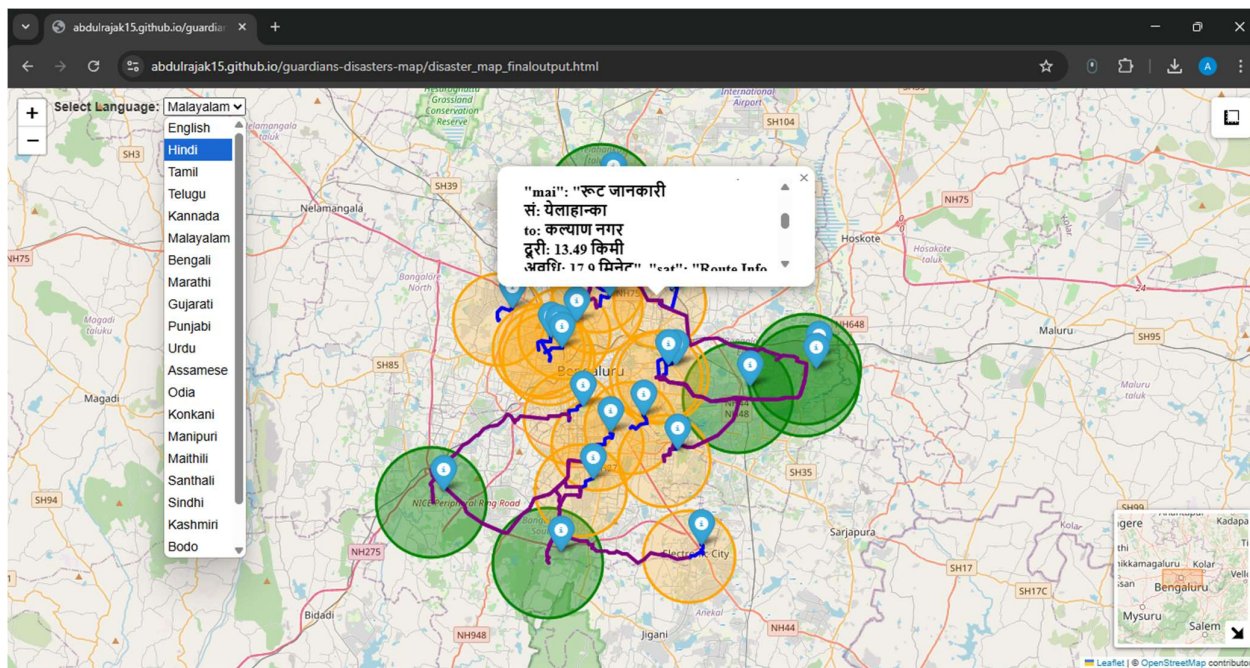


Fig 4.1.7. Language selection dropdown

4.1.8 Mini Compass

Using the **MiniMap plugin**, a live orientation compass is shown:

- Located at bottom corner
- Displays a miniature version of the main map
- Helps identify direction and assists in orientation during navigation

This feature is essential when people need to relocate quickly during floods, cyclones, or earthquakes.

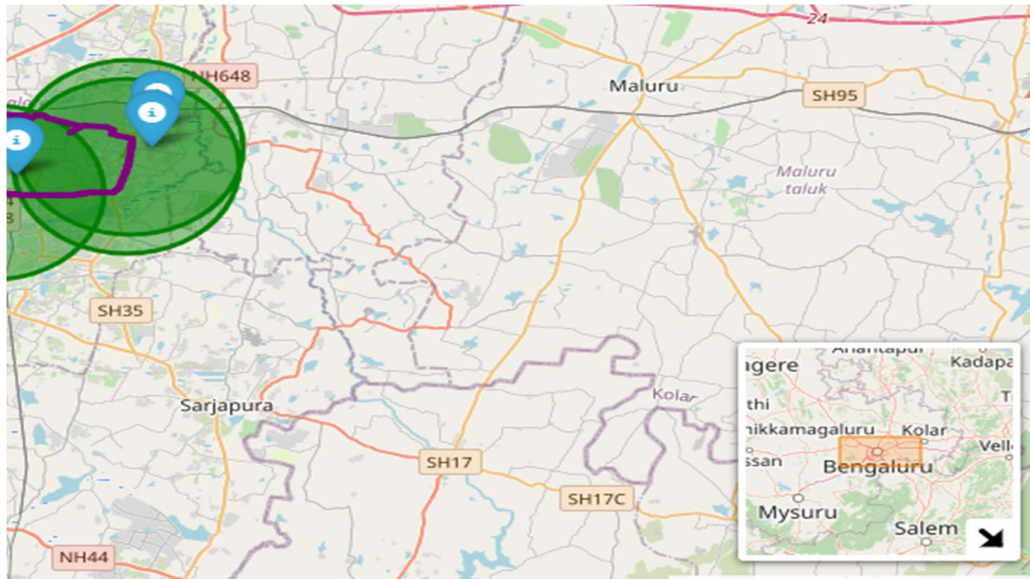


Fig 4.1.8 Mini compass

4.1.9 Distance Measurement Tool:

Integrated MeasureControl plugin allows users to:

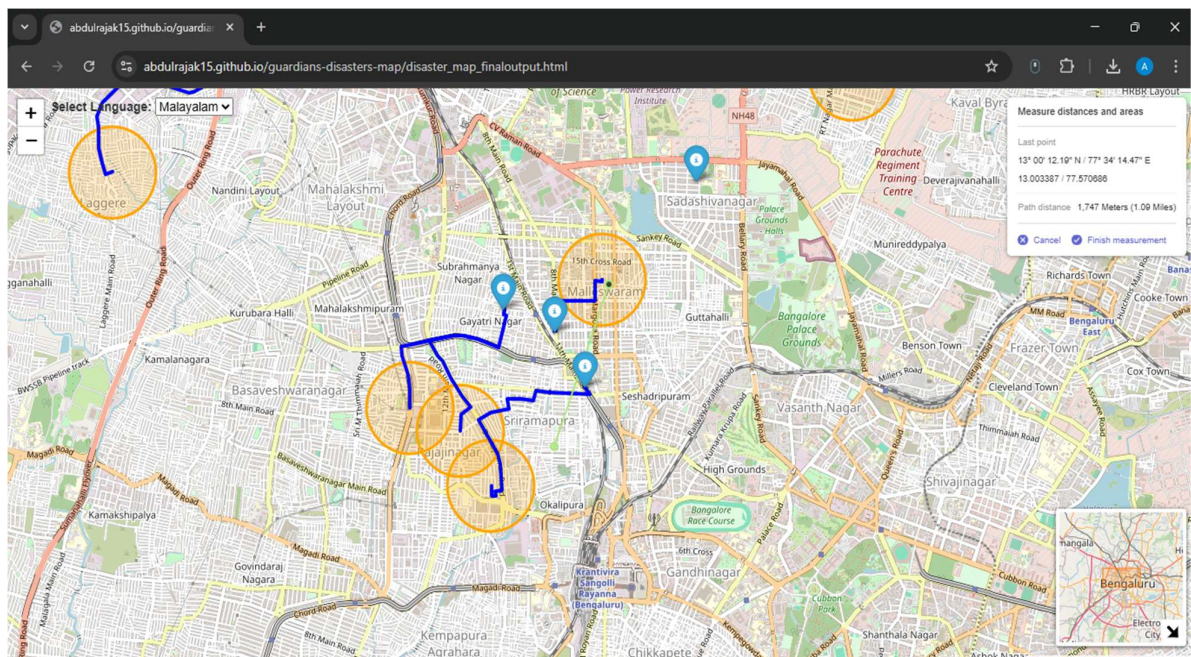


Fig 4.1.9 Distance Measurement Tool

- Click on any two points to **measure the distance**
- View real-time **distance in meters/kilometers**
- Useful for estimating walking/driving time to safety

This is especially helpful for relief workers and first responders to measure optimal paths.

4.1.10 Route Generation Between Zones:

Two types of routing were implemented using **OpenRouteService API**:

- **Danger to Relief Camp Routes:**
 - Automatically calculates **shortest path** from each danger zone to its nearest relief camp.
 - Displayed in **blue polyline** on map.
 - Includes **distance and estimated duration** popup.
- **Safe Zone to Nearby Danger Routes:**

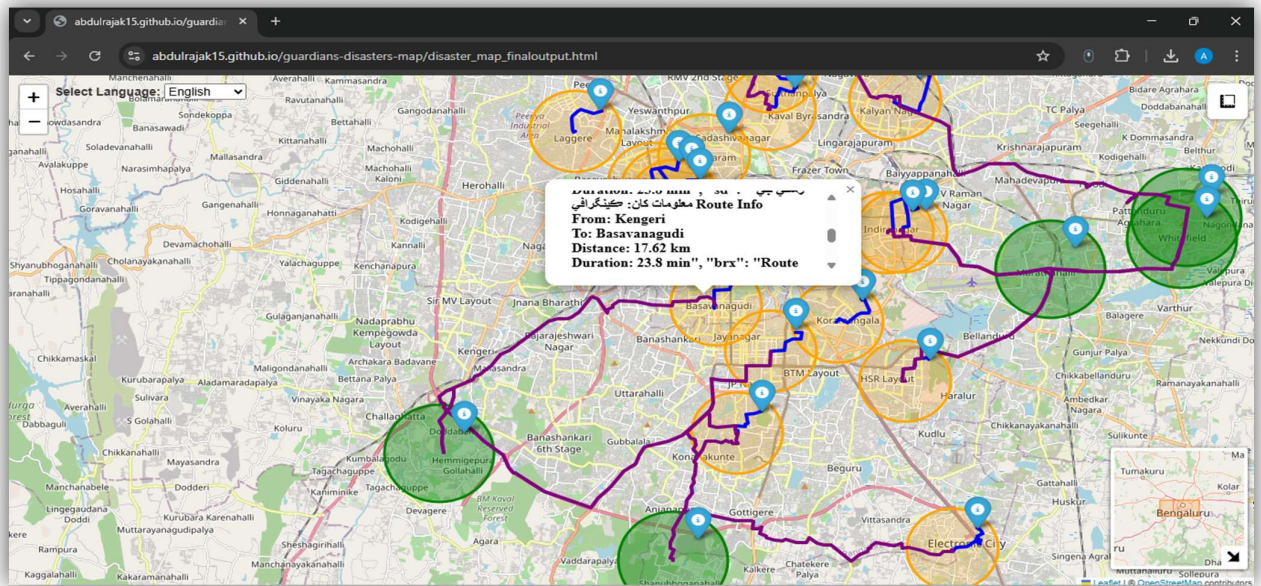


Fig 4.1.10 Route Generation Between Zones

- Identifies top 2–3 closest danger zones from each safe zone.
- Helpful for **rescue volunteers or responders**.
- Displayed in **purple polyline**.

Each route is generated with real-world road data, ensuring **accurate and practical navigation**.

4.1.11 Severity Levels

Every danger zone is classified using a **weighted severity scoring algorithm** based on:

- Wind Speed
- Precipitation
- Soil Moisture
- Relative Humidity

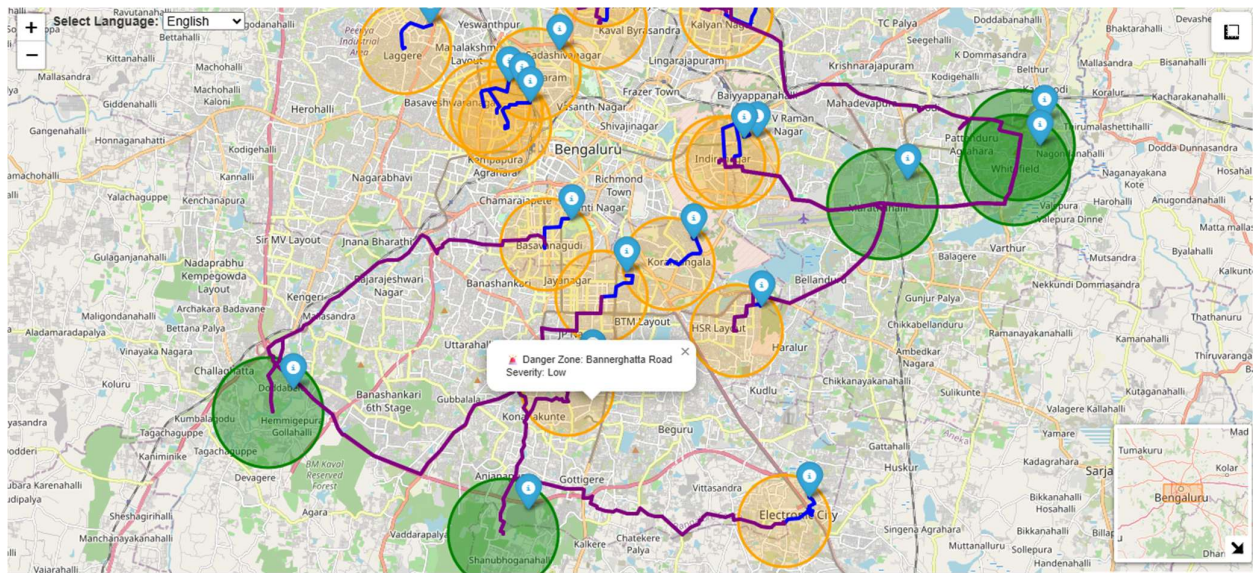


Fig 4.1.11 Severity levels

Severity levels:

Level	Description	Visual
Low	Minor weather risk	Orange
Medium	Moderate threat	Pink
High	Severe emergency	Red

This feature allows **early triage and resource planning**, improving disaster preparedness.

4.1.12 Heat Wave Maps

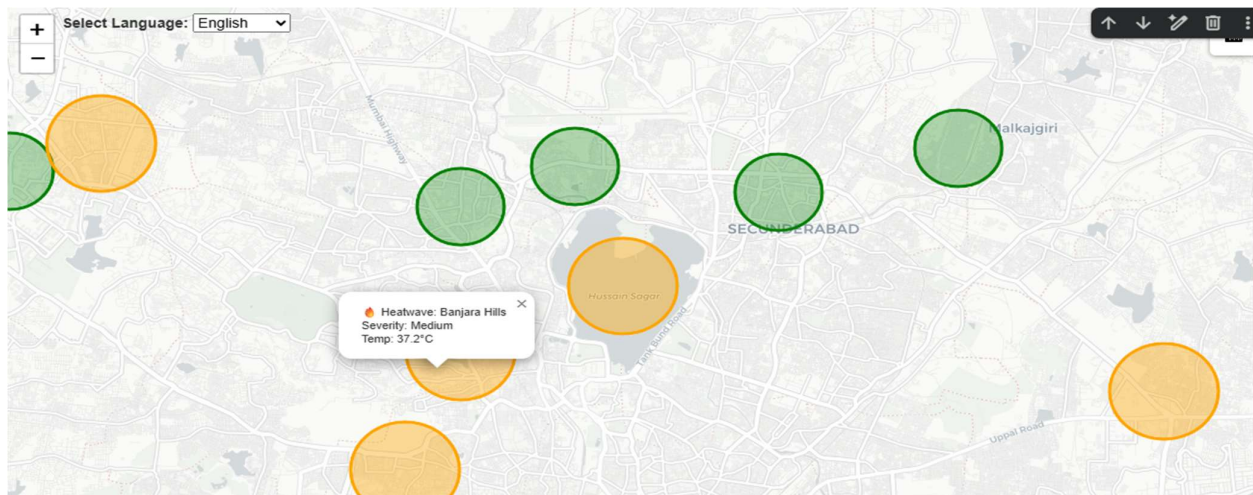


Fig 4.1.12 HeatWave Danger Zone Map

- **Orange Circles → Medium Severity Heatwave**
- **Green Circles → Safe Zones**

These are generated using your **ML prediction (Random Forest / Logistic Regression)**

Based on:

- Temperature
- Heat Index (calculated from humidity + temperature)

4.1.13 HeatWave Safe Zone Map

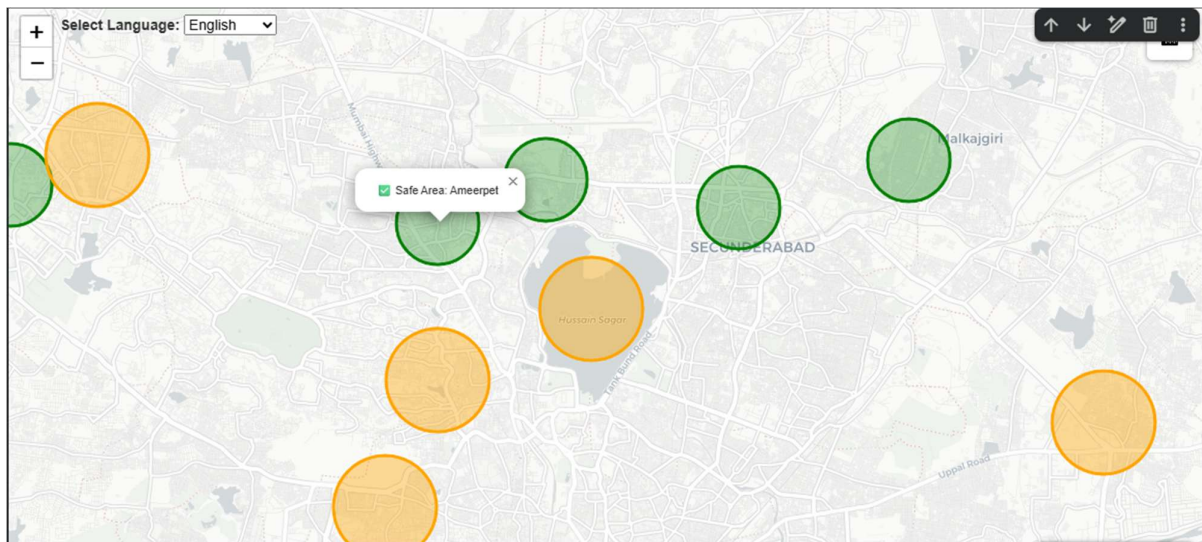


Fig 4.1.13 HeatWave Safe Zone Map

- Green circles represent **Safe Areas**
- Example from your image:
Ameerpet → *Safe Area*

Meaning:

- Temperature + Heat Index are **below risk threshold**
- No immediate heatwave danger.

4.2 Conclusion

The **Multi-Disaster Emergency Response System Using Machine Learning** was conceptualized and implemented with the primary goal of enhancing disaster preparedness and response through the integration of modern technologies, data intelligence, and inclusive communication. The system effectively combines machine learning-based prediction, real-time environmental data analysis,

geospatial visualization, and multilingual support to deliver a comprehensive solution for both citizens and emergency response teams.

Through the integration of Python, machine learning algorithms, Folium, OpenRouteService, translation APIs, and communication services, the project successfully demonstrates how data-driven approaches can be utilized to:

- Detect and classify hazardous zones using machine learning models based on critical environmental parameters such as temperature, precipitation, humidity, wind speed, and soil moisture.
- Handle multiple disaster types, specifically floods and heatwaves, through separate analytical pipelines for improved accuracy and clarity
- Map and visualize danger zones, safe zones, and relief camps using interactive geospatial maps with multilingual popups for enhanced accessibility.
- Provide optimized evacuation routes from flood-affected areas to the nearest relief camps using intelligent routing mechanisms.
- Display heatwave-affected regions separately with severity-based visualization, ensuring focused awareness without confusion.
- Support multilingual communication, enabling users to access critical information in multiple Indian languages
- Enable real-time alerting capabilities through SMS integration, allowing timely communication with individuals in high-risk areas.

One of the key strengths of the system is its ability to move beyond traditional rule-based approaches by incorporating machine learning algorithms such as Random Forest and Logistic Regression. This allows for more accurate and adaptive predictions based on real-time data. Additionally, the dual-map architecture ensures that different disaster types are handled independently, improving usability and decision-making. It can be easily adapted to different regions and enhanced further by integrating additional data sources such as government weather feeds and IoT-based sensors.

In conclusion, this project highlights the potential of combining machine learning, real-time data processing, and geospatial intelligence to transform disaster management systems. It serves not only as a technological innovation but also as a socially impactful solution aimed at improving safety, communication, and resilience in the face of natural disasters.

REFERENCES

1. **Open-Meteo Weather API**
Open-Meteo. (2025). *Weather Forecast API*. Retrieved from <https://open-meteo.com>
2. **OpenRouteService API**
HeiGIT – Heidelberg Institute for Geoinformation Technology. (2025). *OpenRouteService API*. Retrieved from <https://openrouteservice.org>
3. **Folium Python Library**
Folium Contributors. (2025). *Folium Documentation*. Retrieved from <https://python-visualization.github.io/folium/>
4. **Google Translate API via Deep Translator**
Deep Translator. (2025). *Google Translate Integration*. Retrieved from <https://pypi.org/project/deep-translator/>
5. **Twilio SMS API**
Twilio Inc. (2025). *Twilio Programmable Messaging Documentation*. Retrieved from <https://www.twilio.com/docs/sms>
6. **Geopy Python Library**
Geopy Developers. (2025). *Geopy Distance Tools*. Retrieved from <https://geopy.readthedocs.io/>
7. **Pandas Python Library**
The Pandas Development Team. (2025). *Pandas Documentation*. Retrieved from <https://pandas.pydata.org/>
8. **Leaflet.js** (Underlying Map Engine Used in Folium)
Vladimir Agafonkin. (2025). *Leaflet – JavaScript Library for Interactive Maps*. Retrieved from <https://leafletjs.com/>
9. **Government of India – National Disaster Management Authority (NDMA)**
NDMA. (2025). *Guidelines on Disaster Management and Emergency Relief*. Retrieved from <https://ndma.gov.in>
10. Breiman, L. (2001). Scikit-learn Documentation
RandomForestClassifier
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

APPENDIX

```
!pip install scikit-learn
!pip install openrouteservice
!pip install deep_translator
!pip install geopy
!pip install folium
!pip install openpyxl
!pip install twilio
!pip install requests
```

```
dhubri_locations = [
    {"location": "Dhubri Town", "lat": 26.0186, "lon": 89.9856},
    {"location": "Gauripur", "lat": 26.0900, "lon": 89.9500},
    {"location": "Bilasipara", "lat": 26.2400, "lon": 90.2500},
    {"location": "Golokganj", "lat": 26.1200, "lon": 89.8000},
    {"location": "Agomoni", "lat": 26.2600, "lon": 89.7000},
    {"location": "Chapar", "lat": 26.3000, "lon": 90.5000},
    {"location": "Tamarhat", "lat": 26.0500, "lon": 89.7500},
    {"location": "Fakirganj", "lat": 26.3000, "lon": 90.1500},
    {"location": "Sapatgram", "lat": 26.3600, "lon": 90.1000},
    {"location": "South Salmara", "lat": 25.9200, "lon": 89.8500},
    {"location": "Mankachar", "lat": 25.9100, "lon": 89.8200},
    {"location": "Jhagrarpar", "lat": 26.0300, "lon": 89.9000},
    {"location": "Rupsi", "lat": 26.1600, "lon": 89.8500},
    {"location": "Bogribari", "lat": 26.1100, "lon": 90.1000},
    {"location": "Salkocha", "lat": 26.1000, "lon": 90.2500}
]
hyderabad_locations = [
    {"location": "Hitech City", "lat": 17.4435, "lon": 78.3772},
    {"location": "Gachibowli", "lat": 17.4401, "lon": 78.3489},
    {"location": "Madhapur", "lat": 17.4483, "lon": 78.3915},
    {"location": "Kukatpally", "lat": 17.4948, "lon": 78.3996},
    {"location": "Ameerpet", "lat": 17.4375, "lon": 78.4482},
    {"location": "Secunderabad", "lat": 17.4399, "lon": 78.4983},
    {"location": "Begumpet", "lat": 17.4443, "lon": 78.4662},
    {"location": "Banjara Hills", "lat": 17.4126, "lon": 78.4482},
    {"location": "Jubilee Hills", "lat": 17.4239, "lon": 78.4738},
    {"location": "Mehdipatnam", "lat": 17.3924, "lon": 78.4395},
    {"location": "Charminar", "lat": 17.3616, "lon": 78.4747},
    {"location": "Falaknuma", "lat": 17.3300, "lon": 78.4678},
    {"location": "LB Nagar", "lat": 17.3457, "lon": 78.5520},
    {"location": "Uppal", "lat": 17.4058, "lon": 78.5591},
    {"location": "Nagole", "lat": 17.3713, "lon": 78.5695},
    {"location": "Malkajgiri", "lat": 17.4474, "lon": 78.5266}
]
relief_camps = [
    {"name": "Dhubri Safe Zone Camp", "lat": 26.1000, "lon": 90.0500},
    {"name": "Gauripur Elevated Shelter", "lat": 26.1500, "lon": 90.0000},
    {"name": "Bilasipara High Ground Camp", "lat": 26.3000, "lon": 90.3500},
    {"name": "Golokganj Relief Base", "lat": 26.2000, "lon": 89.9000},
    {"name": "Agomoni Safe Shelter", "lat": 26.3200, "lon": 89.8200},
    {"name": "Chapar Emergency Camp", "lat": 26.3800, "lon": 90.6500},
```

```

{"name": "Tamarhat Relief Base", "lat": 26.1500, "lon": 89.8500},
{"name": "Fakirganj Safe Zone", "lat": 26.3800, "lon": 90.3000},
{"name": "Sapatgram Elevated Camp", "lat": 26.4500, "lon": 90.2000},
{"name": "South Salmara Relief Camp", "lat": 26.0500, "lon": 90.0000},
{"name": "Mankachar Safe Shelter", "lat": 26.0500, "lon": 89.9500},
{"name": "Jhagarpar Relief Hub", "lat": 26.1200, "lon": 90.0500},
{"name": "Rupsi Emergency Shelter", "lat": 26.2500, "lon": 90.0000},
{"name": "Bogribari Safe Camp", "lat": 26.2000, "lon": 90.2500},
{"name": "Salkocha Relief Camp", "lat": 26.2000, "lon": 90.4000}
]

import requests
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# =====
# 📌 HEAT INDEX FUNCTION
# =====
def calculate_heat_index(temp, humidity):
    return temp + 0.33 * humidity - 0.7

# =====
# 🌤 WEATHER API (FIXED)
# =====
def get_weather(lat, lon):
    url = f"https://api.open-
meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true&hourly=relative_humidity_
2m,soil_moisture_1_to_3cm,precipitation"

    response = requests.get(url)
    data = response.json()

    current = data["current_weather"]
    index = -1 # latest data

    return {
        "temperature": current["temperature"],
        "wind_speed": current["windspeed"],
        "precipitation": data["hourly"]["precipitation"][index],
        "humidity": data["hourly"]["relative_humidity_2m"][index],
        "soil_moisture": data["hourly"]["soil_moisture_1_to_3cm"][index]
    }

# =====
# 📍 PROCESS LOCATIONS
# =====
def process_locations(locations, mode="flood"):
    data = []

    for loc in locations:
        w = get_weather(loc["lat"], loc["lon"])

```

```

    if mode == "heat":
        heat_index = calculate_heat_index(w["temperature"], w["humidity"])
        data.append({
            "location": loc["location"],
            "lat": loc["lat"],
            "lon": loc["lon"],
            "temperature": w["temperature"],
            "heat_index": heat_index
        })
    else:
        data.append({
            "location": loc["location"],
            "lat": loc["lat"],
            "lon": loc["lon"],
            **w
        })

    return pd.DataFrame(data)

# =====
# 📍 DATASETS
# =====
flood_locations = dhubri_locations
heat_locations = hyderabad_locations

df_flood = process_locations(flood_locations, mode="flood")
df_heat = process_locations(heat_locations, mode="heat")

# =====
# ☐ LABEL CREATION (FIXED)
# =====

# 🌊 Flood Labels (dynamic threshold)
threshold_flood = df_flood[['humidity', 'soil_moisture']].mean().mean()

def classify_flood(row):
    score = row['humidity']*0.5 + row['soil_moisture']*100
    return 1 if score >= threshold_flood else 0

df_flood['label'] = df_flood.apply(classify_flood, axis=1)

# 🔥 Heat Labels (dynamic)
threshold_heat = df_heat['heat_index'].mean()

def classify_heat(row):
    return 1 if row['heat_index'] >= threshold_heat else 0

df_heat['label'] = df_heat.apply(classify_heat, axis=1)

# =====
# 🌊 FLOOD ML MODEL
# =====
X_flood = df_flood[['wind_speed', 'precipitation', 'humidity', 'soil_moisture']]
y_flood = df_flood['label']

```

```

rf_flood = RandomForestClassifier(n_estimators=50, random_state=42)
rf_flood.fit(X_flood, y_flood)

# Safe Logistic Regression
if len(set(y_flood)) > 1:
    lr_flood = LogisticRegression()
    lr_flood.fit(X_flood, y_flood)
    use_lr_flood = True
else:
    use_lr_flood = False

rf_pred_flood = rf_flood.predict(X_flood)
rf_acc_flood = accuracy_score(y_flood, rf_pred_flood)

if use_lr_flood:
    lr_pred_flood = lr_flood.predict(X_flood)
    lr_acc_flood = accuracy_score(y_flood, lr_pred_flood)
else:
    lr_acc_flood = "Skipped (one class)"

best_flood_model = rf_flood

df_flood['ML_Prediction'] = best_flood_model.predict(X_flood)
df_flood['ML_Prediction'] = df_flood['ML_Prediction'].map({0: "Safe", 1: "Danger"})

# =====
# 🔥 HEAT ML MODEL
# =====
X_heat = df_heat[['temperature', 'heat_index']]
y_heat = df_heat['label']

rf_heat = RandomForestClassifier(n_estimators=50, random_state=42)
rf_heat.fit(X_heat, y_heat)

if len(set(y_heat)) > 1:
    lr_heat = LogisticRegression()
    lr_heat.fit(X_heat, y_heat)
    use_lr_heat = True
else:
    use_lr_heat = False

rf_pred_heat = rf_heat.predict(X_heat)
rf_acc_heat = accuracy_score(y_heat, rf_pred_heat)

if use_lr_heat:
    lr_pred_heat = lr_heat.predict(X_heat)
    lr_acc_heat = accuracy_score(y_heat, lr_pred_heat)
else:
    lr_acc_heat = "Skipped (one class)"

best_heat_model = rf_heat

df_heat['ML_Prediction'] = best_heat_model.predict(X_heat)

```

```

df_heat['ML_Prediction'] = df_heat['ML_Prediction'].map({0: "Safe", 1: "Heatwave"})

# =====
# 🏠 OUTPUT
# =====

print("🌊 Flood Accuracy → RF:", rf_acc_flood, " LR:", lr_acc_flood)
display(df_flood[['location', 'precipitation', 'humidity', 'soil_moisture', 'ML_Prediction']])

print("\n🔥 Heatwave Accuracy → RF:", rf_acc_heat, " LR:", lr_acc_heat)
display(df_heat[['location', 'temperature', 'heat_index', 'ML_Prediction']])

import pandas as pd

# =====
# 🏠 Relief Camps
# =====
relief_df = pd.DataFrame(relief_camps)
relief_df['location'] = relief_df['name']
relief_df['zone_status'] = 'Relief Camp'

# =====
# 🌊 FLOOD (Using ML Output)
# =====
safe_flood_df = df_flood[df_flood['ML_Prediction'] == 'Safe'].reset_index(drop=True)
danger_flood_df = df_flood[df_flood['ML_Prediction'] == 'Danger'].copy()

# =====
# 🌊 FLOOD SEVERITY
# =====
def get_score(value, thresholds):
    for i, threshold in enumerate(thresholds):
        if value <= threshold:
            return i
    return len(thresholds)

def calculate_flood_severity(row):
    wind_score = get_score(row['wind_speed'], [7, 10, 20])
    precip_score = get_score(row['precipitation'], [3, 5, 7])
    humidity_score = get_score(row['humidity'], [84, 90, 95])
    soil_score = get_score(row['soil_moisture'], [0.35, 0.5, 0.6])
    return wind_score + precip_score + humidity_score + soil_score

def flood_severity_label(score):
    if score <= 3:
        return 'Low'
    elif score <= 6:
        return 'Medium'
    else:
        return 'High'

danger_flood_df['severity_score'] = danger_flood_df.apply(calculate_flood_severity, axis=1)
danger_flood_df['severity_level'] = danger_flood_df['severity_score'].apply(flood_severity_label)

# =====

```

```

# 🌡️ HEATWAVE (Using ML Output)
# =====
safe_heat_df = df_heat[df_heat['ML_Prediction'] == 'Safe'].reset_index(drop=True)
danger_heat_df = df_heat[df_heat['ML_Prediction'] == 'Heatwave'].copy()

# =====
# 🌡️ HEATWAVE SEVERITY
# =====
def calculate_heat_severity(row):
    # Only based on heat index
    if row['temperature'] < 35:
        return 1
    elif row['temperature'] < 45:
        return 2
    else:
        return 3

def heat_severity_label(score):
    if score == 1:
        return 'Low'
    elif score == 2:
        return 'Medium'
    else:
        return 'High'

danger_heat_df['severity_score'] = danger_heat_df.apply(calculate_heat_severity, axis=1)
danger_heat_df['severity_level'] = danger_heat_df['severity_score'].apply(heat_severity_label)

# =====
# 🏠 OUTPUT
# =====

print("🌊 Flood Danger Zones with Severity")
display(danger_flood_df[['location', 'wind_speed', 'precipitation', 'humidity', 'soil_moisture', 'severity_level']])

print("\n🌡️ Heatwave Zones with Severity")
display(danger_heat_df[['location', 'temperature', 'heat_index', 'severity_level']])

import math

def shift_location(lat, lon, shift_meters_north=1000, shift_meters_east=1000):
    delta_lat = shift_meters_north / 111000
    delta_lon = shift_meters_east / (111000 * math.cos(math.radians(lat)))
    return lat + delta_lat, lon + delta_lon

relief_camps = []
for loc in dhubri_locations:
    shifted_lat, shifted_lon = shift_location(loc['lat'], loc['lon'], 1000, 1000)
    relief_camps.append({
        "name": f"{loc['location']} Relief Camp",
        "lat": shifted_lat,
        "lon": shifted_lon
    })

```

```

for camp in relief_camps:
    print(camp)

import folium
import pandas as pd
from geopy.distance import geodesic
from deep_translator import GoogleTranslator
import openrouteservice
from openrouteservice import convert
import json
from folium.plugins import MiniMap, MeasureControl

client = openrouteservice.Client(key='5b3ce3597851110001cf6248f8d666a0eb9c4ac48f88ae120b3a41d3')

# =====
# 🌐 LANGUAGE SUPPORT
# =====
LANGUAGES = {
    "English": "en", "Hindi": "hi", "Tamil": "ta", "Telugu": "te", "Kannada": "kn",
    "Malayalam": "ml", "Bengali": "bn", "Marathi": "mr", "Gujarati": "gu",
    "Punjabi": "pa", "Urdu": "ur", "Assamese": "as", "Odia": "or"
}

# =====
# 💎 DATA (ONLY FLOOD)
# =====
safe_df = df_flood[df_flood['ML_Prediction'] == 'Safe'].reset_index(drop=True)
danger_df = danger_flood_df.copy()

relief_df = pd.DataFrame(relief_camps)
relief_df['location'] = relief_df['name']

# =====
# MAP CENTER
# =====
avg_lat, avg_lon = df_flood['lat'].mean(), df_flood['lon'].mean()
m = folium.Map(location=[avg_lat, avg_lon], zoom_start=8)

# =====
# LANGUAGE DROPDOWN
# =====
dropdown_html = f"""
<div style="position: fixed; top: 10px; left: 50px; z-index: 1000;">
  <label><b>Select Language:</b></label>
  <select onchange="changeLanguage(this.value)">
    {"".join([f<option value="{code}">{lang}</option>' for lang, code in LANGUAGES.items()])}
  </select>
</div>
<script>
function changeLanguage(lang) {{
  const elements = document.querySelectorAll("[data-translate]");
  elements.forEach(el => {{
    const translations = JSON.parse(el.getAttribute("data-translate"));

```

```

        el.innerHTML = translations[lang] || translations['en'];
    });
    }}
</script>
"""

m.get_root().html.add_child(folium.Element(dropdown_html))

# =====
# TRANSLATION
# =====
def make_translations(text_dict):
    translations = {}
    for _, code in LANGUAGES.items():
        try:
            translations[code] = GoogleTranslator(source='en', target=code).translate(text_dict['en'])
        except:
            translations[code] = text_dict['en']
    return translations

def translated_popup(text_dict):
    translations = make_translations(text_dict)
    escaped = json.dumps(translations).replace('"', '&quot;')
    html = f'<div data-translate="{escaped}">{translations["en"]}</div>'
    return folium.Popup(html, max_width=300)

# =====
# SAFE ZONES
# =====
for _, row in safe_df.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']],
        radius=50,
        color='green',
        fill=True,
        fill_opacity=0.4,
        popup=translated_popup({"en": f"✔ Safe Zone: {row['location']}"})
    ).add_to(m)

# =====
# RELIEF CAMPS
# =====
for _, row in relief_df.iterrows():
    folium.Marker(
        location=[row['lat'], row['lon']],
        icon=folium.Icon(color='blue'),
        popup=translated_popup({"en": f"🏠 Relief Camp: {row['location']}"})
    ).add_to(m)

# =====
# DANGER ZONES
# =====
colors = {'Low': 'yellow', 'Medium': 'orange', 'High': 'red'}

for _, row in danger_df.iterrows():

```

```

folium.CircleMarker(
    location=[row['lat'], row['lon']],
    radius=50,
    color=colors[row['severity_level']],
    fill=True,
    fill_opacity=0.3,
    popup=translated_popup( {
        "en": f"🚰 Flood Zone: {row['location']}<br>Severity: {row['severity_level']}"
    })
).add_to(m)

# =====
# ROUTING FUNCTION
# =====
def get_route_info(start_lat, start_lon, end_lat, end_lon):
    coords = [(start_lon, start_lat), (end_lon, end_lat)]
    route = client.directions(coords)
    decoded = convert.decode_polyline(route['routes'][0]['geometry'])
    duration = route['routes'][0]['summary']['duration']
    distance = route['routes'][0]['summary']['distance']
    return [(lat, lon) for lon, lat in decoded['coordinates']], duration, distance

# =====
# 🚰 DANGER → RELIEF (BLUE)
# =====
for _, danger in danger_df.iterrows():
    d_loc = (danger['lat'], danger['lon'])

    relief_df['distance'] = relief_df.apply(
        lambda x: geodesic(d_loc, (x['lat'], x['lon'])).km, axis=1
    )
    nearest = relief_df.loc[relief_df['distance'].idxmin()]

    try:
        route_coords, duration, dist = get_route_info(
            danger['lat'], danger['lon'],
            nearest['lat'], nearest['lon']
        )

        text = {"en": f"<b>Evacuation Route</b><br>From: {danger['location']}<br>To:
{nearest['location']}<br>Distance: {round(dist/1000,2)} km<br>Time: {round(duration/60,1)} min"}

        folium.PolyLine(route_coords, color='blue', weight=4,
            popup=translated_popup(text)).add_to(m)

    except:
        pass

# =====
# ☐ SAFE → NEAREST DANGER
# =====
for _, safe in safe_df.iterrows():
    safe_loc = (safe['lat'], safe['lon'])

```

```

danger_df['distance'] = danger_df.apply(
    lambda x: geodesic(safe_loc, (x['lat'], x['lon'])).km, axis=1
)
nearest_dangers = danger_df.nsmallest(3, 'distance')

for _, danger in nearest_dangers.iterrows():
    try:
        route_coords, duration, dist = get_route_info(
            safe['lat'], safe['lon'],
            danger['lat'], danger['lon']
        )

        text = {"en": f"<b>Nearby Risk</b><br>From: {safe['location']}<br>To: {danger['location']}<br>Distance: {round(dist/1000,2)} km"}

        folium.PolyLine(route_coords, color='purple', weight=3,
            popup=translated_popup(text)).add_to(m)

    except:
        pass

# =====
# CONTROLS
# =====
m.add_child(MiniMap())
m.add_child(MeasureControl())

# =====
# 🌡 HEATWAVE MAP
# =====

# Data
safe_heat_df = df_heat[df_heat['ML_Prediction'] == 'Safe'].reset_index(drop=True)
danger_heat_df = danger_heat_df.copy()

# Map center (heat only)
avg_lat_h, avg_lon_h = df_heat['lat'].mean(), df_heat['lon'].mean()

map_heat = folium.Map(
    location=[avg_lat_h, avg_lon_h],
    zoom_start=6,
    tiles='CartoDB positron' # avoids 403 error
)

# =====
# LANGUAGE DROPDOWN (SAME)
# =====
dropdown_html_heat = f"""
<div style="position: fixed; top: 10px; left: 50px; z-index: 1000;">
  <label><b>Select Language:</b></label>
  <select onchange="changeLanguage(this.value)">
    {"".join([f"<option value='{code}'>{lang}</option>" for lang, code in LANGUAGES.items()])}
  </select>
</div>

```

```

<script>
function changeLanguage(lang) {{
    const elements = document.querySelectorAll("[data-translate]");
    elements.forEach(el => {{
        const translations = JSON.parse(el.getAttribute("data-translate"));
        el.innerHTML = translations[lang] || translations['en'];
    }});
}}
</script>
"""
map_heat.get_root().html.add_child(folium.Element(dropdown_html_heat))

# =====
# SAFE HEAT ZONES
# =====
for _, row in safe_heat_df.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']],
        radius=40,
        color='green',
        fill=True,
        fill_opacity=0.3,
        popup=translated_popup({"en": f"✔ Safe Area: {row['location']}"})
    ).add_to(map_heat)

# =====
# HEATWAVE ZONES (SEVERITY)
# =====
colors_heat = {'Low': 'yellow', 'Medium': 'orange', 'High': 'red'}

for _, row in danger_heat_df.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']],
        radius=50,
        color=colors_heat[row['severity_level']],
        fill=True,
        fill_opacity=0.4,
        popup=translated_popup({
            "en": f"🔥 Heatwave: {row['location']}<br>Severity: {row['severity_level']}<br>Temp:
{row['temperature']}°C"
        })
    ).add_to(map_heat)

# =====
# CONTROLS
# =====
map_heat.add_child(MiniMap())
map_heat.add_child(MeasureControl())

# =====
# OUTPUT
# =====

print("🌊 Flood Map")

```

```

display(m)

print("🔥 Heatwave Map")
display(map_heat)

# =====
# 📁 SAVE MAPS
# =====

m.save("flood_map.html")
map_heat.save("heatwave_map.html")

# =====
# 📁 DOWNLOAD (COLAB)
# =====

from google.colab import files

files.download("flood_map.html")
files.download("heatwave_map.html")
from twilio.rest import Client

# Replace these with your actual Twilio credentials
account_sid = 'AC9fac488467107b7d9fdf4ce0d39bc2f7'
auth_token = '8e55fea7594f0987ff0a580b4cf9a6b3'
twilio_number = '+12194197902'
# Your friend's verified number (recipient)
to_number = '+919542161644' # e.g., +919876543210

message_body = """
📢 Guardians of Disasters Alert 📢

Check the interactive disaster map here:
https://abdulrajak15.github.io/guardians-of-disasters-map/disaster\_map.html
"""

def send_alert_sms():
    try:
        client = Client(account_sid, auth_token)
        message = client.messages.create(
            body=message_body,
            from_=twilio_number,
            to=to_number
        )
        print(f"✅ Message sent successfully! SID: {message.sid}")
    except Exception as e:
        print(f"❌ Failed to send message: {e}")

if __name__ == "__main__":
    send_alert_sms()

```